

國立陽明交通大學

資訊安全研究所

碩士論文

Institute of Computer and Communications Security

National Yang Ming Chiao Tung University

Master Thesis

Engage-Agent：基於 MITRE Engage 的雙層 LLM Agent

交戰式蜜網系統

Engage-Agent: A 2-Layer MITRE Engage-driven LLM

Agent in a Honeynet

研 究 生：王世揚 (Wang, Shih-Yang)

指導教授：林盈達 (Lin, Ying-Dar)

中華民國 一一五年八月

August 2026

Engage-Agent：基於 MITRE Engage 的雙層
LLM Agent 交戰式蜜網系統
Engage-Agent: A 2-Layer MITRE Engage-driven
LLM Agent in a Honeynet

研 究 生：王世揚
指導教授：林盈達 博士

Student：Shih-Yang Wang
Advisor：Dr. Ying-Dar Lin



Submitted to Institute of Computer and Communications Security
College of Computer Science

National Yang Ming Chiao Tung University
in Partial Fulfillment of the Requirements
for the Degree of
Master of Science
in
Computer Science

August 2026
Taiwan, Republic of China

中華民國 一一五年八月

誌 謝

謝天謝地

王世揚 於

國立陽明交通大學 資訊安全研究所

中華民國 一一五年八月



Engage-Agent：基於 MITRE Engage 的雙層 LLM Agent 交戰式蜜網系統

學生：王世揚

指導教授：林盈達 博士

國立陽明交通大學
資訊安全研究所

摘 要

現代企業攻擊通常包含憑證濫用、權限提升、橫向移動與 Active Directory 滲透等多階段行為。傳統單機蜜罐雖可蒐集攻擊資料，但難以支援跨主機攻擊路徑；既有蜜網雖提供較真實的多主機環境，卻多以靜態或被動觀察為主，缺乏根據攻擊者行為動態調整環境的能力。近期大型語言模型（Large Language Models, LLMs）雖被應用於蜜罐回覆生成，但此類方法通常直接產生攻擊者可見輸出，缺乏與真實系統狀態的一致性，因而限制了長程互動與多階段攻擊誘導效果。

為解決上述問題，本研究提出 Engage-Agent，一套基於 MITRE Engage 的雙層 LLM Agent 交戰式蜜網系統。系統以真實 AD-domain 蜜網作為基礎，並透過環境修改而非直接回覆生成來進行欺敵。Strategist LLM 負責分析攻擊者近期指令、推論攻擊意圖並選擇交戰策略與工具；Planner LLM 則將策略轉換為結構化工具參數，透過 MCP 與 Ansible 修改真實蜜網環境。此設計將高階攻擊意圖理解與低階環境部署分離，使系統能在保持執行可靠性的同時，動態建立檔案、帳號、憑證、服務與跨主機線索，引導攻擊者往更深層的攻擊路徑前進。

實驗結果顯示，本系統能有效提升攻擊互動的真實性與深度。互動真實性方面，本系統的 Perceived Real-System Rate 達到 80%，分別比 QRASSH 與 LLM-based Honey-pot 高出 61 與 48 個百分點。攻擊深度方面，本系統的平均最大攻擊深度相較 QRASSH 提升約 5.2 倍，且相較 LLM-based Honey-pot 提升約 71%。此外，固定情境 Planner benchmark 顯示，fine-tuned two-layer 設計的 composite score 相較 one-layer 提升約 67%，並相較未微調 two-layer 提升約 18%。

綜上所述，本研究證明，以真實多主機環境為基礎的環境驅動式欺敵能提升攻擊互動真實性與深度，而雙層 LLM Agent 則能提升環境修改計畫的品質與適應性。此系統為結合大型語言模型、MITRE Engage 與企業級蜜網環境的主動防禦技術提供一個可行架構。

關鍵字：蜜網、自適應欺敵、大型語言模型、MITRE Engage、Active Directory、MCP、環境修改

Engage-Agent: A 2-Layer MITRE Engage-driven LLM Agent in a Honeynet

Student : Shih-Yang Wang Advisor: Dr. Ying-Dar Lin

Institute of Computer and Communications Security
National Yang Ming Chiao Tung University

Abstract

Modern enterprise attacks often involve multi-stage behaviors such as credential abuse, privilege escalation, lateral movement, and Active Directory compromise. Traditional single-host honeypots cannot effectively support cross-host attack progression, while existing honeynets are often static or passive. Recent LLM-based honeypots improve response generation, but directly fabricated replies are not grounded in persistent system state.

This thesis proposes Engage-Agent, a two-layer MITRE Engage-driven LLM agent for adaptive honeynet engagement. Built on a real AD-domain honeynet, Engage-Agent performs deception through environment modification rather than direct reply generation. The Strategist LLM infers attacker intent and selects engagement tools, while the Planner LLM converts these decisions into structured tool arguments executed through MCP and Ansible. This design separates high-level reasoning from execution-oriented planning and enables bounded, realistic environment modification.

Experimental results show that Engage-Agent improves interaction realism and attack depth. Ours achieves an 80% Perceived Real-System Rate, outperforming QRASSH and the LLM-based Honeypot by 61 and 48 percentage points, respectively. Ours also achieves approximately $5.2\times$ higher average maximum attack depth than QRASSH and 71% higher depth than the LLM-based Honeypot. In the Planner benchmark, the fine-tuned two-layer design improves the composite score by 67% over the one-layer design and by 18% over the non-fine-tuned two-layer design.

Overall, this thesis demonstrates that environment-driven deception in a real multi-host honeynet can improve attacker interaction realism and depth, while the two-layer LLM agent improves the quality of adaptive environment-modification plans.

Keywords: Honeynet, Adaptive Deception, Large Language Model, MITRE Engage, Active Directory, MCP, Environment Modification

Contents

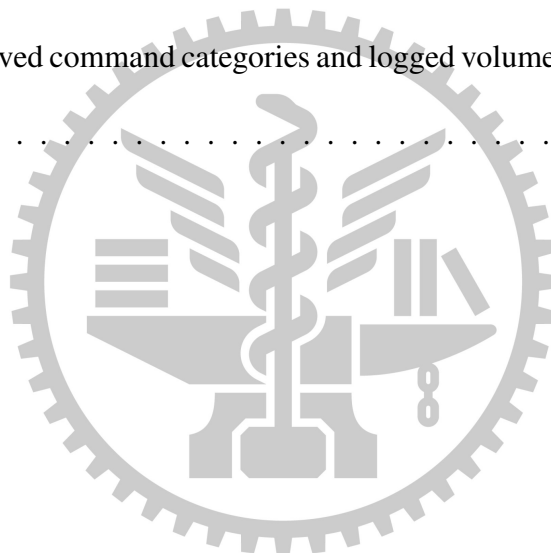
Chinese Abstract	i
English Abstract.....	ii
Contents	iii
List of Figures	v
List of Tables.....	vii
1 Introduction.....	1
2 Background and Related Work	4
2.1 Honeypot and Honeynet.....	4
2.2 MITRE ATT&CK and MITRE Engage.....	4
2.3 LLM Agent and MCP	5
2.4 Active Directory.....	6
2.5 Relate Work	7
3 Problem Formulation	10
3.1 Notations	10
3.2 Problem Statement	11
4 Solution Approach	13
4.1 Solution Overview	13

4.2	Strategist LLM and Tool Selection	15
4.3	Planner LLM and MCP Toolkit	16
4.4	AD Domain Environment Design	19
5	Implementation	21
5.1	System Architecture	21
5.2	Dataset Generation and Fine-Tuning	23
5.3	MCP Toolkit Generation and Execution	26
5.4	Attacker Interaction Simulation	27
6	Experimental Evaluation	30
6.1	Experimental Setup	30
6.2	Attack Progression and Behavioral Diversity	31
6.3	Interaction Realism and Cross-Host Behavior	33
6.4	Planner Output Quality Benchmark	36
6.5	Online Deployment Observations	38
6.6	Discussion and Limitations	40
7	Conclusions and Future Work	42
	References	44

List of Figures

Figure 1	MCP Server	6
Figure 2	Active Directory	7
Figure 3	Solution overview	14
Figure 4	Role of the Strategist LLM in attacker-intent inference and tool selection .	16
Figure 5	Tool-specification-guided plan generation and script rendering by the Plan- ner LLM and MCP toolkit	19
Figure 6	AD-domain-based honeynet environment with role-specific hosts and ser- vices	20
Figure 7	Runtime architecture of the implemented Engage-Agent system.	22
Figure 8	Offline dataset generation pipeline for Strategist supervision construction and Planner fine-tuning.	23
Figure 9	Semi-automated generation and validation process for MCP tools.	26
Figure 10	End-to-end attacker interaction simulation loop.	28
Figure 11	Maximum attack stage depth across 30 controlled attacker sessions. Higher values indicate deeper simulated attack progression.	32
Figure 12	Number of unique techniques observed across 30 controlled attacker ses- sions. Higher values indicate broader behavioral coverage.	33

Figure 13	Perceived Realism Rate in the controlled attacker experiments. Higher values indicate that command responses are judged as more plausible real-system interaction.	34
Figure 14	Lateral movement attempts and success rates in the controlled attacker experiments. Attempt volume and success rate should be interpreted jointly. . .	35
Figure 15	Planner output quality comparison under fixed scenario-stage inputs. The benchmark evaluates internal plan quality rather than end-to-end attacker engagement.	37
Figure 16	Online SSH audit activity observed at the deployed proxy entry point. . .	39
Figure 17	Observed command categories and logged volume from online SSH proxy traffic.	40



List of Tables

Table 1	Survey on recent research in adaptive honeypots and honeynets	8
Table 2	Notations	10
Table 3	Summary of MCP tool groups and their alignment with selected MITRE	
	Engage activities	18
Table 4	Dataset construction scale in the offline supervision pipeline	24
Table 5	Planner supervised fine-tuning configuration	25
Table 6	Compared systems in the controlled attacker experiments	31

Chapter 1. Introduction

Modern enterprise attacks, such as lateral movement, credential reuse, and Active Directory compromise, are inherently multi-stage and span across multiple hosts, requiring realistic environments to study attacker behavior progression. Honeypots, as security deception systems designed to attract attackers and collect behavioral data, have been widely used in controlled environments for this purpose. However, traditional honeypots typically simulate only a single host, making them insufficient for capturing multi-stage, cross-host attack progression. As a result, they suffer from limited attack depth and incomplete observation of attacker behavior progression, restricting their effectiveness in analyzing complex enterprise-scale attacks.

To address these limitations, honeynet environments have been proposed, where multiple interconnected honeypots form a multi-host network that better reflects real enterprise infrastructures. Such environments enable the observation of cross-host attack behaviors and improve realism compared to single-host honeypots. Nevertheless, existing honeynet research, such as HoneyFactory[1] and ADLAH[2], primarily focuses on environment construction and dynamic deployment. These systems largely operate in a passive observation mode, lacking mechanisms for active interaction with attackers. Consequently, they are unable to influence attacker decisions or guide attack progression, which limits attack depth, reduces behavioral diversity, and weakens engagement effectiveness. This reveals a fundamental gap between environment realism and adaptive interaction capability in current honeynet systems.

In parallel, adaptive honeypot research has introduced dynamic interaction capabilities using reinforcement learning (RL) and large language model (LLM)-based decision mechanisms. Systems such as QRASSH[3] and HoneyGPT[4] demonstrate the ability to generate engagement strategies and interact with attackers in real time. However, these approaches are designed for single-host settings and cannot be directly extended to honeynet environments. A direct combination of multiple independent adaptive honeypots results in independent decision mak-

ing without global coordination, failing to manage cross-host attack progression. Extending adaptive interaction to honeynets introduces significant challenges, including a large decision space, complex multi-host system states, and the need for cross-host consistency. More importantly, there is currently no unified framework that enables coordinated, environment-level, multi-host engagement. Therefore, a key challenge is to design an adaptive honeynet system that simultaneously achieves enterprise-level realism and globally coordinated attacker interaction.

To address this challenge, this thesis proposes an adaptive honeynet engagement framework that integrates global decision making with environment-level deception. Instead of generating simulated reply content, the proposed approach dynamically modifies the underlying system environment across multiple hosts, ensuring that attacker interactions emerge naturally from actual system state changes. The framework employs an LLM-based decision agent that analyzes attacker commands, infers attacker intent, and generates coordinated engagement strategies. These strategies are executed through environment-level actions across multiple hosts, enabling guided attack progression. Through this design, the system achieves deeper attack exploration, higher behavioral diversity, and realistic multi-host interactions.

The main contributions of this work are summarized as follows:

- We propose an adaptive honeynet engagement framework that bridges the gap between environment realism and adaptive attacker interaction.
- We introduce an environment-driven deception approach that performs system-level modifications, enabling attacker interactions to emerge from real system behavior rather than simulated replies.
- We design an MITRE Engage-aligned engagement strategy that maps attacker intent to structured deception activities, bridging the gap between high-level adversary engagement goals and executable environment-level actions.

The remainder of this thesis is organized as follows. Chapter 2 presents the background and related work. Chapter 3 introduces the problem formulation. Chapter 4 describes the

proposed solution approach. Chapter 5 details the system implementation. Chapter 6 presents experimental results. Finally, Chapter 7 concludes the thesis and discusses future work.



Chapter 2. Background and Related Work

2.1 Honeypot and HoneyNet

Honeypots are security deception systems that simulate a single host, while honeynets extend this concept by emulating an entire network environment. Both aim to safely attract attackers and analyze their behaviors and techniques. In practice, open-source tools such as Cowrie enable the deployment of high-interaction static honeypots, and platforms like T-Pot allow multiple honeypots to be deployed together as a static honeynet. Commercial deception technologies, including Zscaler Deception and Proofpoint Identity Threat Defense, deploy decoys within real enterprise environments to lure attackers. Detected malicious activities are then redirected to controlled sandbox environments for observation.

However, existing solutions primarily focus on capturing attacks rather than actively engaging with attackers. They lack mechanisms to dynamically interact with and guide attackers toward deeper and more diverse attack behaviors, which limits their value for comprehensive security analysis. This work aims to fill this gap by enabling active and adaptive engagement within honeynet environments.

2.2 MITRE ATT&CK and MITRE Engage

MITRE ATT&CK[5] is a widely used knowledge base for characterizing adversary tactics, techniques, and procedures. It is primarily designed to support attack analysis, threat intelligence, and detection engineering by describing attacker behavior in a structured form. However, ATT&CK focuses on modeling attacker actions and does not explicitly provide guidance on how defenders should respond during adversary engagement. Therefore, although ATT&CK is useful for interpreting attack behavior, it is not sufficient for guiding adaptive deception in this work.

To address this limitation, MITRE proposed MITRE Engage[6], a framework specifically designed for active defense, deception, and adversary engagement. MITRE Engage has been designed with the explicit goal of enabling defenders to interact with attackers more effectively and collect richer behavioral data. The framework organizes defender knowledge into three layers: goals, approaches, and activities, thereby linking high-level engagement objectives to concrete defensive actions. Compared with ATT&CK, which emphasizes attack representation, Engage is more suitable for this thesis because it provides a clearer basis for selecting and executing engagement actions in a honeynet.

Accordingly, this work adopts MITRE Engage as the conceptual foundation for engagement strategy design. The proposed system uses this framework to guide the selection and execution of environment-level deception activities that dynamically influence attacker behavior within the honeynet.

2.3 LLM Agent and MCP

LLM-based agents are AI decision systems driven by large language models, capable of reasoning, planning, and invoking external tools to perform actions. They typically operate in a perception–reasoning–action loop, where the agent analyzes environmental context, generates decisions, and executes actions through integrated tools.

In the context of this work, directly generating executable Ansible scripts using LLMs is unreliable, as free-form generation often leads to invalid syntax, inconsistent configurations, and low execution success rates. Instead of allowing the LLM to generate scripts from scratch, this thesis adopts a tool-based approach, where environment modifications are performed through a predefined set of executable tools. Each tool encapsulates a verified Ansible template and exposes a structured interface, enabling the LLM to generate only the necessary parameters rather than full scripts. This design significantly improves execution stability and ensures that all actions are grounded in valid system operations.

To support structured tool interaction, the Model Context Protocol (MCP) provides a standardized interface that connects LLM agents with tools and environments. As illustrated in Figure 1, MCP abstracts multiple external services into a unified interface, allowing the LLM

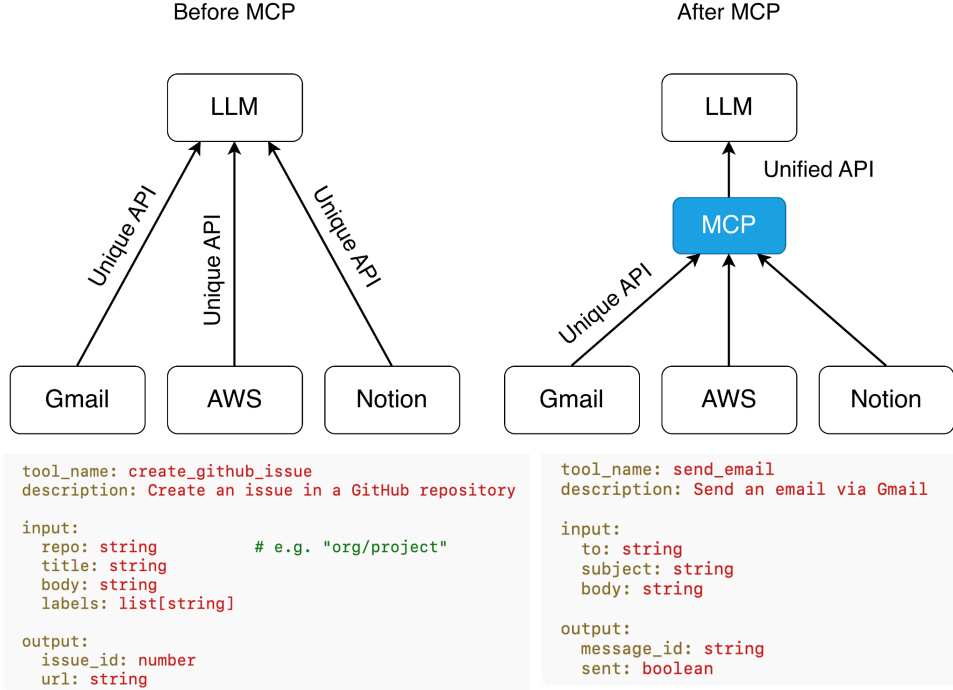


Figure 1: MCP Server

to interact with diverse systems through a single API layer. In this work, MCP is adopted as a practical implementation choice for defining tool interfaces and managing execution, rather than a strict requirement of the system design. The key contribution lies in the tool-based execution paradigm, while MCP serves as a convenient and widely-used framework to realize this interaction model.

2.4 Active Directory

Active Directory (AD) is a directory service developed by Microsoft for centralized management of users, hosts, authentication, authorization, and security policies in Windows-based enterprise environments. In real organizations, AD commonly serves as the core infrastructure that connects accounts, endpoints, servers, and administrative control within a domain, as shown in Figure 2. Because of this central role, AD is not only widely used in enterprise networks but also strongly shapes how attackers move, escalate privileges, and access resources in practice.

This importance also makes AD a high-value attack target. Once attackers compromise an AD environment, they can often expand access across multiple hosts, abuse trust relationships,

and gain broad control over enterprise resources. As a result, many real-world attack techniques are closely tied to AD, including credential abuse, lateral movement, and privilege escalation. Representative examples such as Pass-the-Hash and Kerberoasting further show that AD is not merely a background service, but a major operational surface in enterprise attacks.

For this reason, this work adopts AD as the foundation of the honeynet environment. Since adaptive deception in this work is evaluated through multi-step enterprise attack behavior, the experimental environment must expose the same identity structure, trust relationships, and privilege dependencies that attackers encounter in real networks. An AD-based honeynet therefore provides a realistic and representative basis for evaluating whether the proposed system can influence attacker progression under practical enterprise attack conditions.

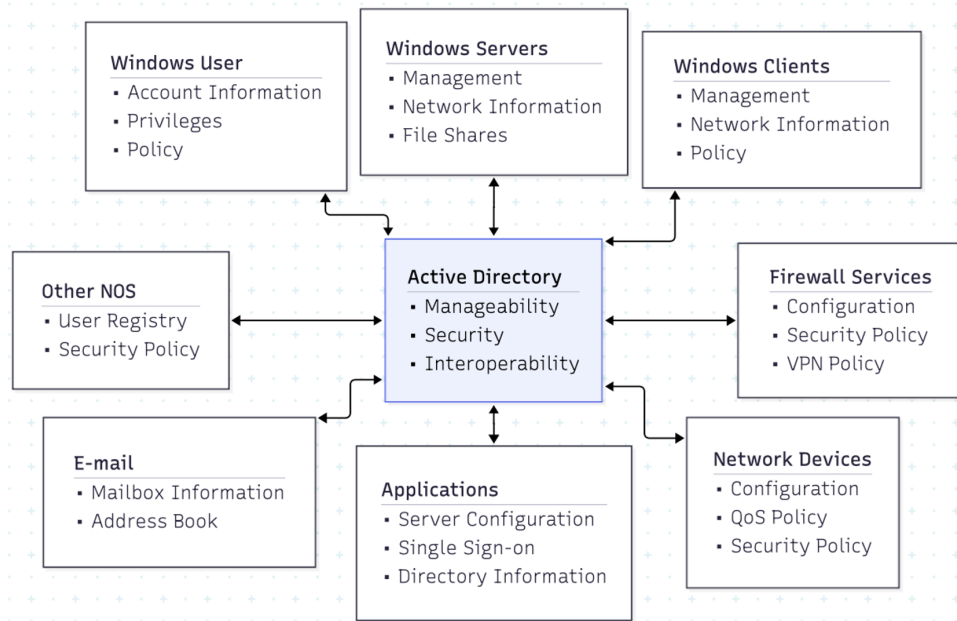


Figure 2: Active Directory

2.5 Relate Work

Existing research on adaptive deception systems can be broadly categorized into two directions: adaptive honeypots and adaptive honeynets. These two lines of work focus on different aspects of attacker interaction and environment design.

Adaptive honeypot research primarily focuses on dynamically engaging with attackers at the single-host level. Representative systems such as QRASSH[3], HoneyGPT[4], and Se-

Table 1: Survey on recent research in adaptive honeypots and honeynets

Category	Paper	Real	AD	Objective			Method	Action Design
				Depth	Realism	Diversity		
Honeypot	QRASSH [3]	–	–	✓	–	–	RL	Response manipulation
	HoneyGPT [4]	–	–	✓	✓	✓	LLM	Response generation
	Senior [7]	✓	✓	✓	✓	✓	RL	Response manipulation
Honeynet	HoneyFactory [1]	✓	–	–	✓	–	Rule-based	Honeynet generation
	ADLAH [2]	–	–	–	✓	–	RL	Honeypot deployment
	HoneyICS [8]	✓	–	–	✓	–	Rule-based	ICS network emulation
	Mirage [9]	✓	✓	–	–	–	Rule-based	File-system modification
	Ours	✓	✓	✓	✓	✓	LLM	MITRE Engage-guided environment modification

nior[7] introduce reinforcement learning (RL) or large language model (LLM)-based decision mechanisms to model attacker commands and generate responses. These systems typically operate at the command or reply level, selecting actions from a limited action space to influence attacker behavior. Their primary objective is to prolong attacker engagement and collect richer behavioral data. However, due to their single-host nature, these systems are limited in their ability to capture multi-stage attacks involving lateral movement and cross-host interactions.

In contrast, adaptive honeynet research focuses on constructing realistic multi-host environments that emulate enterprise infrastructures. Systems such as HoneyFactory[1], ADLAH[2], HoneyICS[8], and Mirage[9] represent this direction. HoneyFactory[1] dynamically generates honeynet environments with enterprise-like topologies to capture multi-stage attacks across multiple hosts. ADLAH[2] applies reinforcement learning to optimize honeypot placement and redirects anomalous connections to backend systems such as Cowrie, improving attack capture efficiency. HoneyICS[8] targets industrial control system (ICS) environments by simulating physical processes and IoT devices to capture domain-specific attacks. Mirage[9] adopts an environment-level deception approach by modifying file system artifacts using rule-based strategies to influence attacker perception and behavior. To systematically compare existing approaches, Table 1 summarizes representative adaptive honeypot and honeynet systems in terms of engagement capability, environment realism, and behavioral coverage. While these systems improve environmental realism and enable multi-stage attack observation, they generally rely on static or rule-based strategies and lack mechanisms for adaptive, semantic-level interaction with attackers.

As shown in Figure 1, adaptive honeypot systems emphasize interaction intelligence, lever-

aging RL or LLM-based decision mechanisms to dynamically respond to attackers. However, these systems primarily operate at the reply level and are limited to single-host environments, resulting in restricted support for lateral movement and reduced enterprise realism. Consequently, they cannot effectively model complex multi-stage attack scenarios.

In contrast, honeynet systems focus on environment realism by constructing multi-host infrastructures and supporting environment-level artifact manipulation. These systems enable the observation of cross-host attack behaviors and improve realism compared to single-host honeypots. However, as indicated in Table 1, their strategies are typically static or rule-based, focusing on deployment optimization or attacker disruption rather than adaptive engagement driven by semantic reasoning. As a result, their ability to actively influence attacker behavior and guide attack progression remains limited.

These observations reveal a fundamental gap between interaction intelligence and environment realism. Existing approaches do not provide a unified framework that integrates dynamic attacker engagement with enterprise-scale environments. In particular, prior work lacks the combination of semantic-aware reasoning, coordinated multi-host decision making, and environment-level execution. This gap motivates the need for an adaptive honeynet framework that combines LLM-based decision making with environment-level manipulation to enable realistic and guided multi-stage attack progression. To address this gap, this thesis proposes Engage-Agent, which is detailed in the following chapters.

Chapter 3. Problem Formulation

This chapter formulates adaptive deception in a honeynet as a sequential, multi-host decision and deployment problem. Unlike traditional adaptive honeypots that mainly manipulate reply content on a single host, our setting requires the defender to reason over attacker behavior across multiple interaction steps and coordinate executable environment-level modifications over multiple hosts in a persistent enterprise-like honeynet. The goal is not merely to generate plausible responses, but to steer the attacker toward deeper and more diverse attack progression through realistic system-state manipulation while preserving execution reliability and real-time responsiveness.

3.1 Notations

Table 2: Notations

Categories	Notation	Description
Interaction & Session	$C_i, i \in \{1, \dots, N_c\}$	The i -th command issued by the attacker
	$X_i = (C_{i-k+1}, \dots, C_i)$	Recent attack command context observed at interaction step i
	R_i	Response observed by the attacker after executing C_i
	N_c	Number of commands in one engagement session
	k	command context window size
Honeynet & Environment	$\mathcal{H} = \{H_1, \dots, H_{N_H}\}$	Set of hosts in the honeynet
	$H_j, j \in \{1, \dots, N_H\}$	A honeynet host (DC, web, mail, client, etc.)
	N_H	Number of computer host in the honeynet
LLM Agent & Planning	a_i	Engagement strategy selected by the Strategist LLM at interaction step i
	T_i	Subset of MCP tools selected by the Strategist LLM at interaction step i
	$a_i, T_i = f_{\text{strat}}(C_i, T_i)$	Maps command context and available tools to strategy and selected tools
	$P = (p_1, p_2, \dots, p_{N_H}) = f_{\text{plan}}(a_i, T_i, T_s)$	Generated execution plans by the Planner LLM, $p_j = \emptyset$ if no action is assigned to host H_j
MCP & Execution	T_l	set of available MCP tools
	T_s	set of MCP tool specifications
	$S = (s_1, s_2, \dots, s_{N_H}) = \text{Render}(P)$	Executable scripts rendered from plans P
	$E = (e_1, e_2, \dots, e_{N_H}) = \text{Exec}(S)$	Execution results of scripts S

The main notations used in this chapter are summarized in Table 2. These symbols describe the problem from four aspects: attacker–honeynet interaction, the multi-host honeynet environment, the two-layer LLM decision process, and MCP-based execution.

Specifically, C_i , X_i , R_i , and N_C are used to represent the sequential interaction between the attacker and the honeynet. $\mathcal{H} = \{H_1, \dots, H_{N_H}\}$ denotes the set of honeynet hosts. Given the recent command context X_i , the Strategist LLM produces a high-level strategy action and selected tools, and the Planner LLM further generates host-level plans $P = (p_1, \dots, p_{N_H})$. These plans are then rendered into executable scripts S and produce execution results E . Together, these notations formalize the end-to-end process from attacker interaction to environment-level deception execution.

3.2 Problem Statement

We formulate adaptive deception in a honeynet as a sequential decision problem over a persistent multi-host environment. At each interaction step i , the defender observes the recent attacker command context X_i and must determine how to modify the honeynet so that the attacker perceives realistic and coherent system feedback that steers attack progression toward deeper exploration.

The problem can be formulated as follows:

- **Input**

- X_i : Recent attack command context observed at interaction step i

- **Output**

- R_i : Attacker-observed response induced by the execution result E of the generated plan P at step i

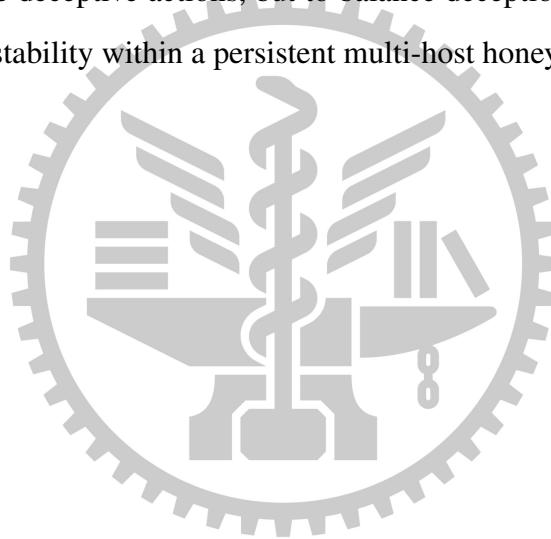
- **Objective**

- Maximize attack stage progression: $\max_{1 \leq i \leq N_c} s_i$, where $s_i \in \{1, 2, \dots, 7\}$ denotes the attack stage reached at step i

- **Constraints**

- Execution reliability: $\frac{\sum_{i=1}^{N_H} \mathbf{1}(e_i = \text{success})}{N_H} > 0.9$
- Real-time response constraints: $t_{\text{strategist}} + t_{\text{planner}} + t_{\text{execution}} \leq 90 \text{ (sec)}$

This problem introduces several fundamental challenges in applying adaptive deception to a persistent multi-host honeynet. First, there is an inherent tension between semantic reasoning and execution stability. Although LLMs provide strong contextual understanding and can infer attacker intent from command sequences, unconstrained free-form generation may produce outputs that are ambiguous, inconsistent, or difficult to execute reliably in real systems. As a result, the system must translate high-level semantic reasoning into constrained, tool-based actions that remain operationally stable. Second, the problem is complicated by multi-host coordination. Unlike single-host deception settings, an engagement strategy in a honeynet must affect multiple hosts in a coherent manner, while preserving cross-host and cross-step state consistency over time. Actions applied to one host may influence the plausibility and behavior of others, making coordinated planning substantially more difficult. Therefore, the core challenge is not only to generate deceptive actions, but to balance deception effectiveness, enterprise realism, and execution stability within a persistent multi-host honeynet.



Chapter 4. Solution Approach

This chapter presents the proposed solution to the adaptive honeynet engagement problem formulated in Chapter 3. The goal is to build an LLM-based cybersecurity framework that operates on a real-environment honeynet, monitors attacker behavior, and modifies the underlying environment to deceive the attacker and guide subsequent attack progression. Unlike traditional adaptive honeypots, which mainly manipulate replies on a single host, the proposed system is designed for a persistent multi-host setting in which attacker-visible responses emerge from actual system behavior after coordinated environment-level modifications.

4.1 Solution Overview

Figure 3 illustrates the overall workflow of the proposed adaptive honeynet engagement framework. The system consists of five main components: the attacker, the SSH proxy, the LLM agent, the defender MCP toolkit, and the AD domain environment. The attacker interacts with the honeynet through the SSH entry point, while the proxy serves as the interception point that captures commands and maintains recent command context. This context is then forwarded to the LLM agent, which is composed of a Strategist LLM and a Planner LLM.

As shown in Figure 3, at interaction step i , the recent attacker commands form the command context X_i . The Strategist LLM takes X_i as input, infers attacker intent, determines the engagement direction, and selects suitable tools from the available tool set T_l . The resulting strategy a_i and selected tools T_i are then passed to the Planner LLM, which converts them into an executable plan P by following the tool specifications T_s defined in the defender MCP toolkit. The MCP toolkit further renders P into executable scripts S and executes them in the AD domain environment across one or more hosts. The attacker subsequently observes response R_i , which is produced by the modified real environment rather than by directly fabricated replies. In this way, the system implements the Chapter 3 formulation as a sequential

mapping from command context to strategy and tool selection, then to executable planning and script rendering, and finally to attacker-observed response.

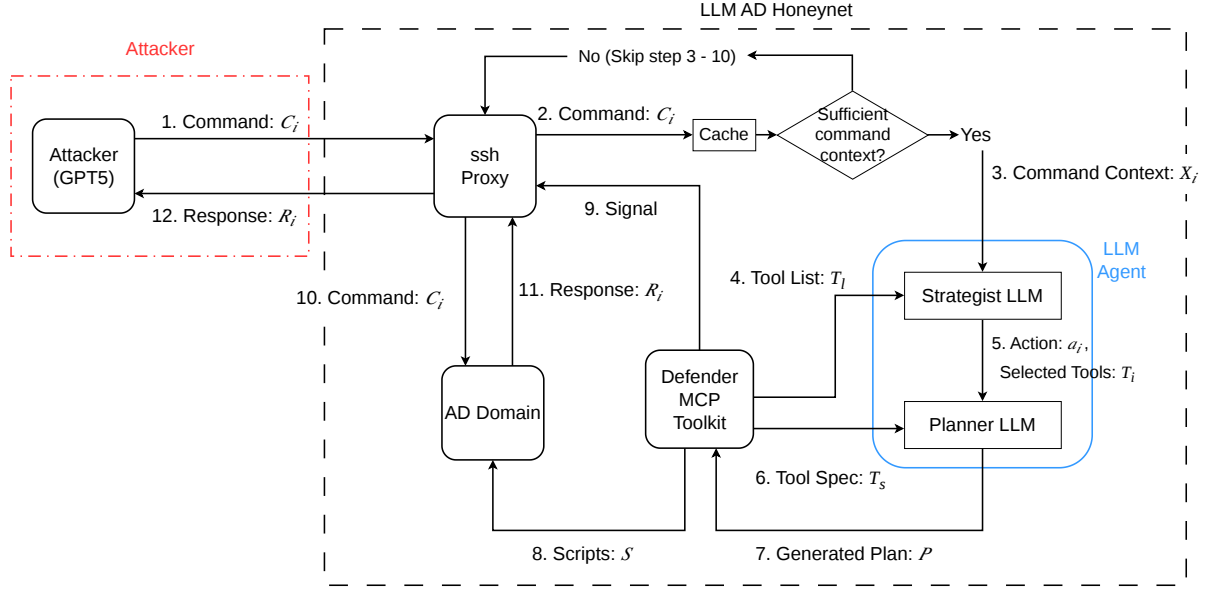


Figure 3: Solution overview

This design increases attack depth through the coordinated roles of the Strategist, the Planner, and real environment modification. The Strategist analyzes recent command context rather than isolated commands, allowing the system to infer the attacker’s current objective and choose engagement actions that match the current attack direction. The Planner and MCP toolkit then turn this high-level decision into dynamic but reliable modifications of the real honeynet, such as exposing files, accounts, credentials, services, or cross-host clues that are relevant to the attacker’s next step. As a result, the framework does not merely produce plausible replies, but continuously reshapes the environment to present believable opportunities that keep the attacker engaged and encourage progression toward later attack stages. By grounding later observations in actual system-state changes, this design achieves both persistent consistency and deeper attack progression.

Another key design choice is the use of two LLMs under practical local-LLM constraints. In an interactive honeynet, the agent must perform repeated reasoning with low operational cost and stable latency, making local deployment more practical than relying on a large remote model for every step. Under this constraint, a single-model design is too broad: one model would need to simultaneously understand attacker behavior, decide engagement direction, se-

lect tools, and prepare deployment-ready outputs. The proposed framework therefore separates these roles into Strategist LLM and Planner LLM. The Strategist focuses on contextual attacker understanding and high-level engagement decision, while the Planner focuses on stable generation of execution-ready tool parameters. This separation makes each model address a narrower task, reducing inference complexity and improving controllability under local deployment constraints.

4.2 Strategist LLM and Tool Selection

Effective adaptive honeynet engagement requires the defender to infer the attacker’s current objective before deciding the next engagement action. This intent cannot be reliably determined from a single observed command, because the attacker’s goal is usually revealed by several recent commands taken together. As a result, immediate reaction to one command can easily misinterpret the attacker’s current direction. This also makes conventional reinforcement-learning-style state abstraction less suitable for the present problem, since the defender must first compress multi-command behavior into a fixed handcrafted state before any action can be selected. In a real honeynet, however, attacker behavior is open-ended, context-dependent, and semantically rich, making such manual state design difficult and brittle. The proposed framework therefore uses an LLM at this stage, allowing the system to reason directly over recent command context and infer what the attacker is currently trying to achieve before deciding how to engage.

As illustrated in Figure 4, the Strategist LLM takes recent command context X_i and the available MCP tool set as input, and converts them into a structured high-level engagement decision. It first infers attacker intention, so that the system can identify the attacker’s current objective rather than merely react to surface-level commands. It then generates engagement strategy a_i , which specifies the defender-side direction for the next step. Finally, it selects tools T_i from the available tool set, binding the strategy to a concrete subset of defender capabilities while keeping the decision separate from later deployment details. The output of this stage is therefore not a direct environment modification, but a structured decision consisting of attacker intention, engagement strategy, and selected tools.

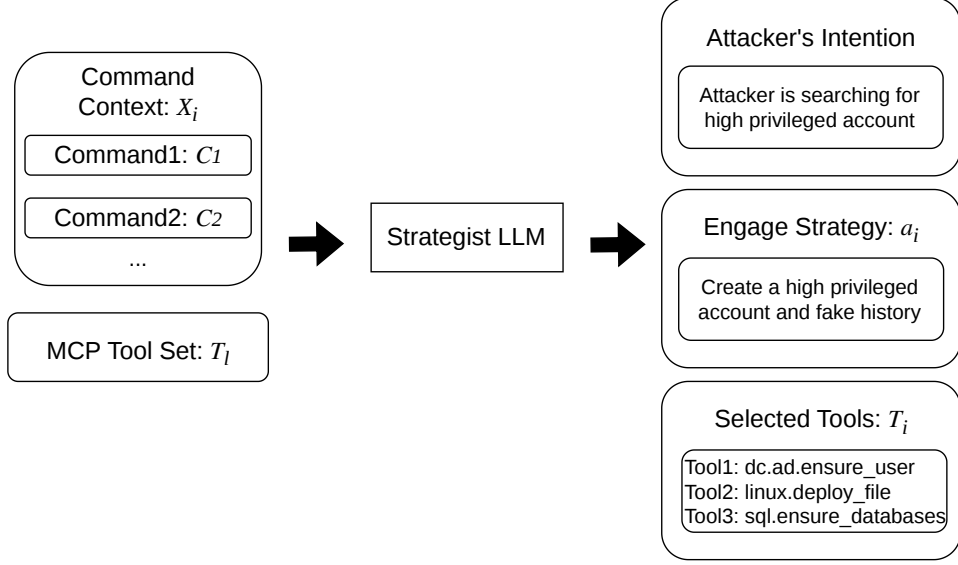


Figure 4: Role of the Strategist LLM in attacker-intent inference and tool selection

Assigning tool selection to the Strategist keeps the choice of defender capabilities tightly coupled with attacker understanding. If this responsibility were deferred to the Planner stage, the high-level engagement decision would be split across two modules, weakening the connection between attacker interpretation and defender response. At the same time, the Strategist stops at selecting tools and does not directly operate them or generate deployment scripts. Direct execution at this stage would mix attacker-oriented reasoning with host-specific deployment details, increase the burden of the first reasoning stage, and make the overall pipeline harder to control. The Strategist therefore decides which defender capabilities should be used, while the next stage determines how those capabilities should be instantiated in the environment. Because this stage primarily requires flexible semantic interpretation rather than highly regular structured output, prompt engineering is more appropriate than fine-tuning. Fine-tuning a small model on a fixed pattern of attacker contexts would constrain the Strategist’s ability to generalize across diverse attack scenarios, which is the opposite of what this stage requires.

4.3 Planner LLM and MCP Toolkit

After the Strategist determines the engagement strategy and selects tools for the next step, the defender must decide how these choices should be realized in the honeynet. Prior adaptive deception systems[3][4][10][7] often realize defender decisions by directly modifying or gen-

erating attacker-visible replies. This approach is flexible at the response level and can quickly create the desired attacker perception. However, in a honeynet environment, reply-level manipulation is fundamentally limited because later attacker actions may verify files, accounts, services, permissions, or cross-host relationships that were never actually changed. This inconsistency is especially problematic in a persistent multi-host setting, where the attacker can revisit earlier observations and validate them through follow-up commands. For this reason, the proposed framework chooses environment modification rather than direct reply fabrication. By changing the real environment, the system preserves cross-step consistency while still indirectly shaping the replies that the attacker later observes.

Choosing environment modification introduces a different requirement: the deployment method must remain dynamic enough to adapt to different attacker contexts while also remaining reliable enough to execute correctly in the real honeynet. Fixed scripts provide stable execution, but their adaptability is limited because one predefined script cannot cover the variety of attacker goals and contexts encountered across sessions. Direct script generation by an LLM offers higher flexibility, but often produces unstable low-level outputs, incorrect tool usage, and hard-to-debug script errors. The proposed framework therefore uses Planner LLM together with the MCP toolkit to balance dynamic planning and reliable deployment.

The MCP toolkit is designed as a bounded deception capability space rather than an arbitrary collection of host-specific scripts. Its tool set is derived from selected MITRE Engage activities and from the environment-modification opportunities available in the AD domain. Among the three Engage goals, namely Expose, Affect, and Elicit, this work focuses on Affect activities because they most directly correspond to concrete modifications of the honeynet environment that can influence attacker progression. Expose activities mainly support observation, detection, and intelligence collection, which are handled separately by the SSH proxy and logging layer in the proposed system. Elicit activities, in contrast, aim at longer-term behavioral inducement and are therefore beyond the scope of the single-step engagement actions considered in this work.

Accordingly, each MCP tool represents a bounded defender capability for modifying the environment, such as introducing decoy identities, planting artifacts, creating credential surfaces, leaving breadcrumbs, or inducing controlled failure. Each tool exposes explicit input

fields and usage constraints, allowing the Planner to assign concrete arguments to predefined capabilities rather than generate unrestricted deployment logic. Table 3 summarizes this design by grouping the MCP tools according to their primary deception function and mapping each group to selected MITRE Engage activities. This bounded structure makes planning more controllable, improves execution reliability, and keeps the deployed actions aligned with the engagement goals of the proposed system.

Table 3: Summary of MCP tool groups and their alignment with selected MITRE Engage activities

Tool Group	#Tools	Inspired from Engage Affect Activity						
		Lures	Personas	Artifact Diversity	Information Manipulation	Introduced Vulnerabilities	Network Manipulation	Email Manipulation
Identity and Account Exposure	3	V	V		V			
Credential Bait	3	V		V	V			V
Artifact and Data Decoys	11	V		V				
Breadcrumb and Misdirection	5	V			V		V	
Privilege Escalation Lures	2	V				V		
Service and Endpoint Lures	3	V					V	V
Controlled Failure Induction	7				V	V		V
Cross-host Movement Bait	1	V			V		V	

Planner LLM and MCP toolkit implement this design as a bounded deployment pipeline. As illustrated in Figure 5, the Planner takes engagement strategy a_i , selected tools T_i , and MCP tool specifications as input. Based on this information, it generates a structured plan P that specifies concrete tool arguments rather than low-level deployment scripts. The MCP toolkit then renders P into executable scripts S through predefined templates and deploys them to the honeynet. In this way, planning remains context-aware, while execution remains bounded by tool schemas and verified rendering logic.

The Planner stage is a constrained output problem rather than an open-ended reasoning problem. Its main requirement is stable and repeatable generation of structured tool parameters that satisfy tool schemas and remain consistent with the selected strategy and tools. For this reason, the Planner is fine-tuned instead of relying on prompt engineering alone. Supervised fine-tuning (SFT) is used to train the model to map planning context to tool arguments. Each training example pairs planner-side input, including engagement strategy, and selected tools, with the corresponding tool parameters. Following a teacher-student distillation pattern, a stronger LLM is used as a teacher model to generate planning labels, which are curated into an SFT dataset for training the smaller, locally deployed Planner model. This design makes the

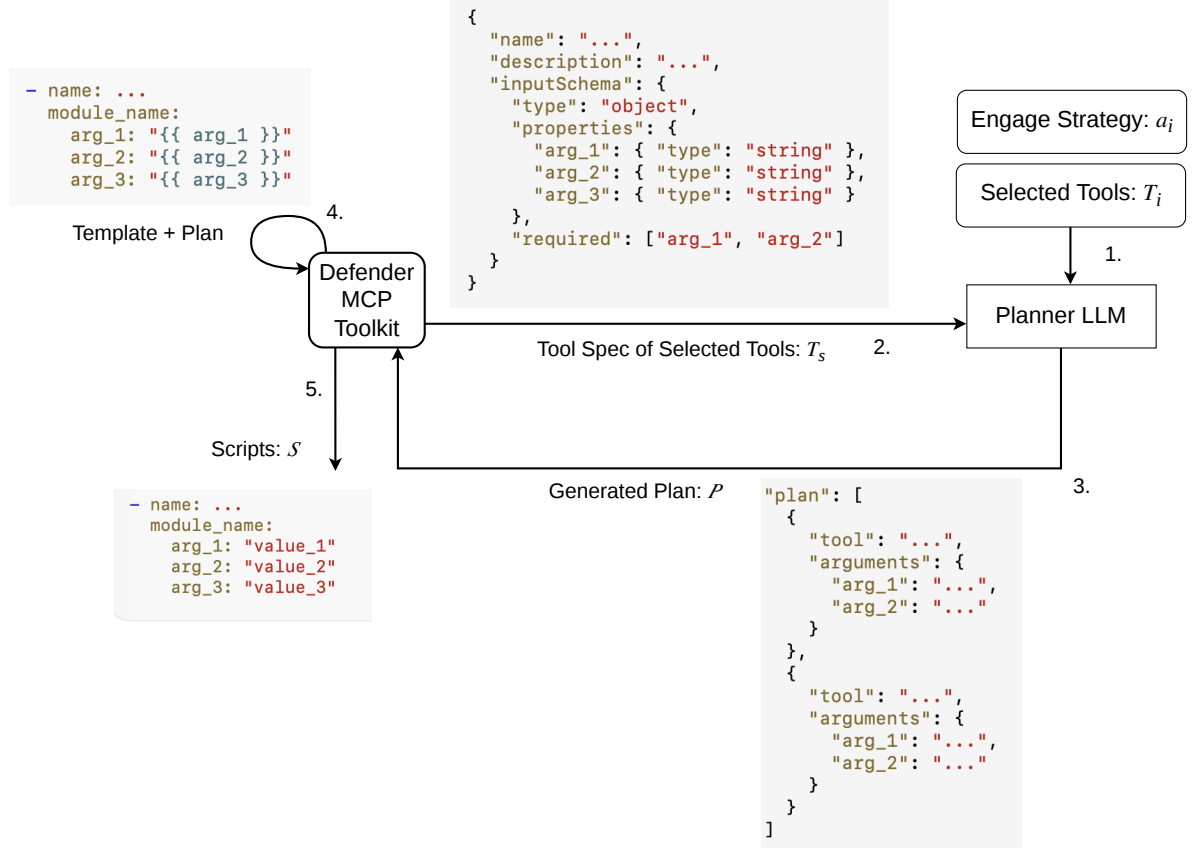


Figure 5: Tool-specification-guided plan generation and script rendering by the Planner LLM and MCP toolkit

Planner more reliable at producing schema-compliant, complete, and deployment-ready parameters.

4.4 AD Domain Environment Design

Environment-level deception cannot rely on a single host or text-only reply generation. Later-stage attacker behaviors such as credential access, privilege escalation, and lateral movement require persistent identities, privilege relationships, service diversity, and multiple connected hosts. A single-host environment cannot provide these conditions in a realistic way, and fabricated replies alone are difficult to sustain once the attacker begins to verify files, accounts, services, or host relationships. For this reason, the proposed framework is built on a real multi-host environment in which defender modifications can create believable cross-host attack opportunities.

As shown in Figure 6, the framework adopts an AD-based honeynet as its environment substrate. The environment includes a domain controller, Windows hosts, Linux hosts, and service hosts such as web, SQL, and mail servers. Each host role contributes a different attacker-visible surface: the domain controller provides identity and privilege structure, Windows hosts provide user and endpoint context, Linux hosts provide shell and file-system interaction surfaces, and service hosts provide application-oriented attack surfaces. As a result, the framework gains multiple places where defender tools can plant decoys, clues, credentials, and controlled obstacles that are consistent with realistic enterprise behavior.

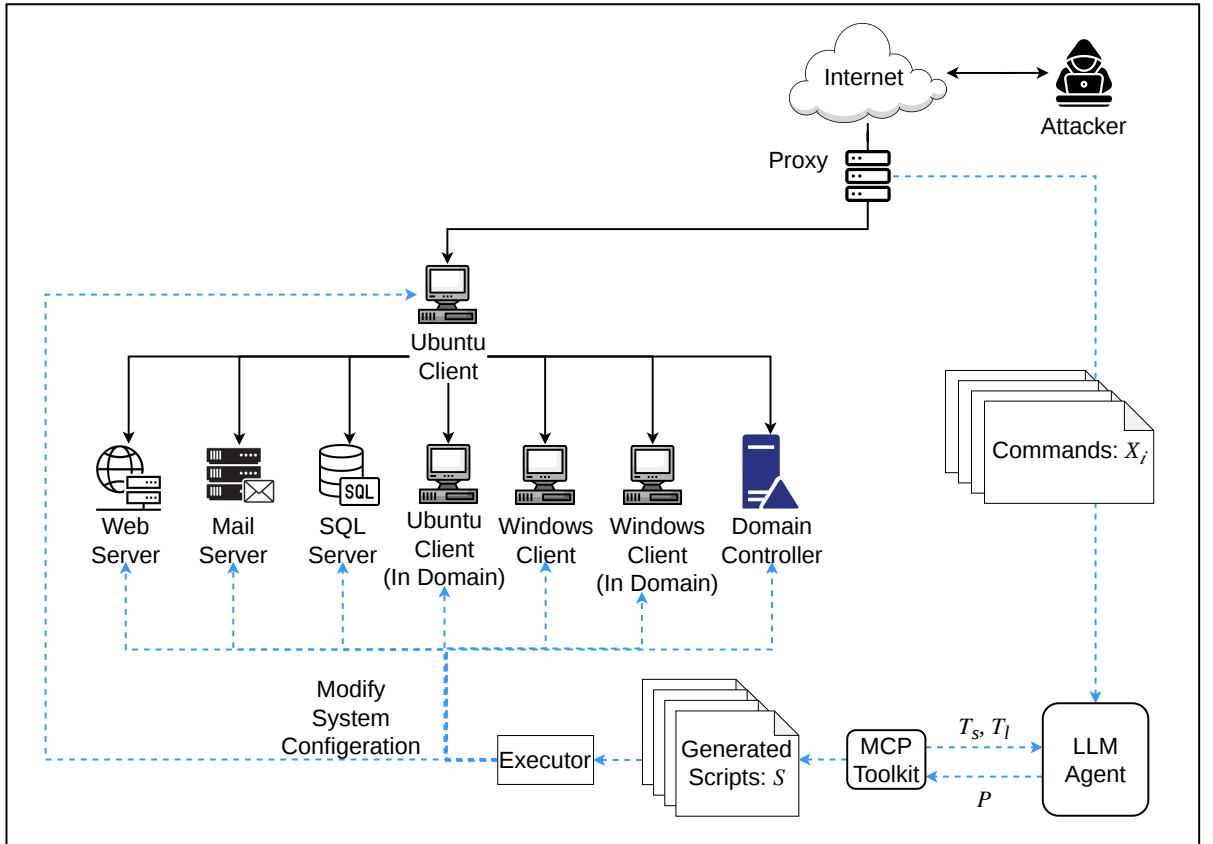


Figure 6: AD-domain-based honeynet environment with role-specific hosts and services

This AD-based multi-host design better matches real enterprise attack progression than a generic lab environment or a single-host honeypot. Because later attacker observations are grounded in actual environment changes rather than fabricated replies, cross-host deception becomes more meaningful and more sustainable over time. The AD domain therefore serves not only as the deployment target of defender actions, but also as the enterprise-like substrate that supports deeper attack progression.

Chapter 5. Implementation

This chapter describes the implementation of the proposed Engage-Agent system. Section 5.1 presents the runtime system architecture and explains how the attacker runtime, SSH interception process, backend planning-and-execution service, and AD-domain honeynet are integrated. Section 5.2 describes the offline dataset generation and Planner fine-tuning pipeline. Section 5.3 explains how the MCP toolkit is generated, validated, and executed through Ansible-based environment modification. Finally, Section 5.4 presents the LLM-driven attacker interaction simulator used to evaluate the system under adaptive attack behavior.

5.1 System Architecture

Figure 7 shows the runtime architecture of the implemented Engage-Agent system. The implementation is divided into four runtime parts: the attacker runtime, the frontend SSH interception process, the backend planning-and-execution service, and the AD-domain honeynet. The attacker runtime maintains an interactive SSH session and generates attack commands from prior interaction history. The frontend process is a Paramiko-based MITM SSH proxy that accepts attacker traffic, extracts commands, accumulates recent command context, and invokes the Strategist LLM when the context threshold is reached. The backend service is implemented as a self-designed agent runtime rather than by directly using an existing agent framework. It exposes a FastAPI-based HTTP REST interface, receives Strategist outputs from the frontend, runs Planner-side inference, and executes MCP-based environment modifications. The AD-domain honeynet contains the live hosts and services modified by the backend and observed by the attacker. This separation reflects the actual deployment boundary of the system: SSH interception, LLM inference, backend execution, and honeynet modification are implemented as distinct runtime layers.

The attacker-facing path is implemented through one persistent SSH session between the

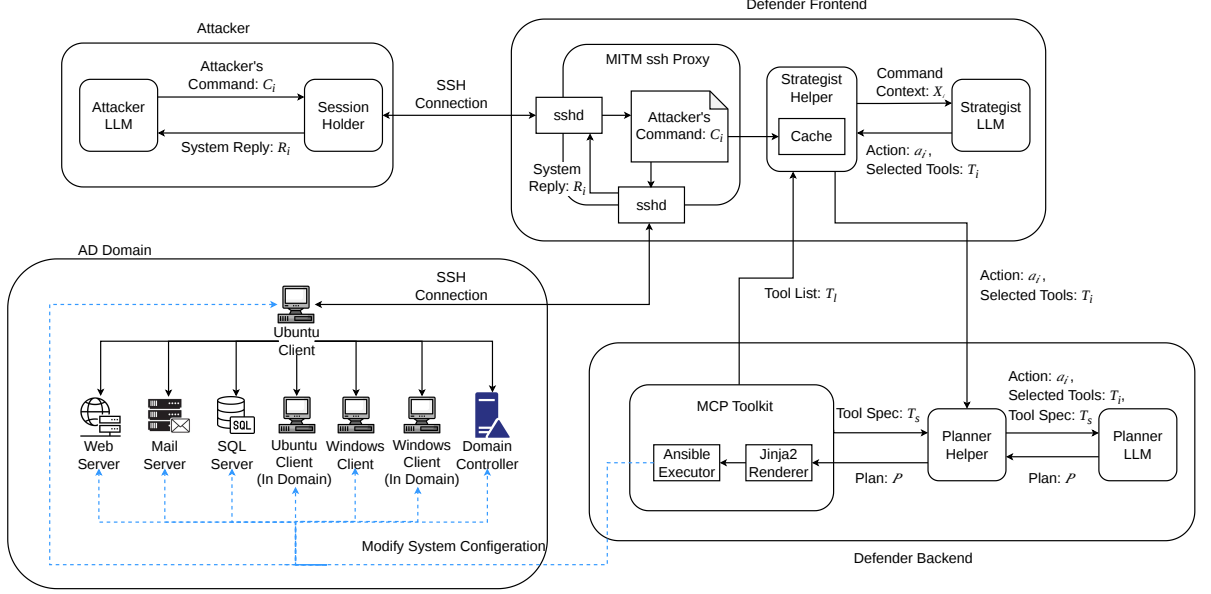


Figure 7: Runtime architecture of the implemented Engage-Agent system.

attacker runtime and the frontend proxy. At each step, the Attacker LLM generates a command C_i from the session history, and the session holder sends it through the existing SSH connection. The attacker simulator uses GPT-5-mini to generate open-ended attack commands, while the defender-side Strategist uses a local Qwen3-8B model and the Planner uses the fine-tuned local Phi-4 model described in Section 5.2. The MITM proxy forwards each command to the Ubuntu entry host in the AD domain while recording the command stream and session metadata. Intercepted commands are buffered as recent command context X_i and passed to the Strategist-side process once the batch threshold of $k = 8$ commands is met. The frontend communicates with the backend through the HTTP REST interface rather than through MCP; MCP is used inside the backend as the tool-execution abstraction for environment modification. This design keeps the attacker interaction continuous while allowing the defender to analyze command context asynchronously within the same session.

The backend path starts when the frontend forwards the Strategist output, including engagement strategy a_i and selected tools T_i , to the backend API. The Strategist LLM is responsible for attacker-intent inference and high-level tool selection, while the Planner LLM converts the selected strategy and tools into executable tool arguments. In the implemented system, the Planner is the fine-tuned local model described in Section 5.2, while the Strategist remains prompt-driven. The Planner pipeline combines the Strategist output with MCP tool specifica-

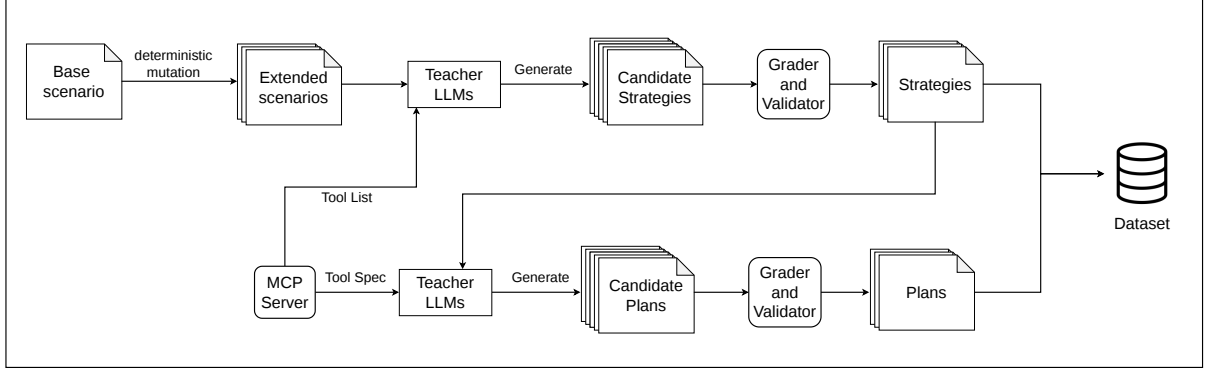


Figure 8: Offline dataset generation pipeline for Strategist supervision construction and Planner fine-tuning.

tions T_s and generates a structured plan P . The MCP execution module validates the plan, binds tool arguments to Jinja2-based Ansible templates, renders playbooks, and executes them through the Ansible executor. The resulting modifications are applied to the corresponding AD-domain hosts through the inventory, and their effects appear to the attacker as ordinary target-host behavior in later SSH interactions.

5.2 Dataset Generation and Fine-Tuning

Figure 8 illustrates the offline supervision pipeline used to construct the Planner fine-tuning dataset. The pipeline starts from five manually designed attack scenarios tailored to the AD-domain honeynet. Each scenario represents a common attack path over the deployed hosts and services and contains 24 attacker commands. Because the runtime system invokes the LLM after accumulating a short recent command context, each scenario is divided into three stage-level segments of eight commands. To increase coverage while preserving the same attack logic, each base scenario is expanded into 20 variants through deterministic mutation, in which environment-dependent fields (target hosts, user accounts, file paths, service names, credentials) are systematically replaced from a predefined pool while the attack logic and command order are preserved. This produces 100 scenario variants and 300 stage-level samples. These samples serve as the common input units for supervision generation and match the command granularity used during runtime inference.

Teacher-model supervision is generated in two stages. For the 300 stage-level samples,

Table 4: Dataset construction scale in the offline supervision pipeline

Stage	Count	Description
Base scenarios	5	Manually designed AD-domain attack scenarios
Scenario variants	100	5 scenarios $\times$ 20 controlled variants
Stage-level samples	300	100 variants $\times$ 3 stages
Strategist candidates	1200	300 samples $\times$ 4 teacher LLMs
Selected Strategist outputs	300	One selected Strategist output per sample
Planner candidates	1200	300 Planner inputs $\times$ 4 teacher LLMs
Selected Planner outputs	300	One validated Planner output per sample
Final SFT examples	300	Planner feature-label pairs

four teacher LLMs independently generate Strategist candidates, producing 1200 candidates in total. Each candidate contains attacker profiling, reasoning over the recent command context, an engagement strategy, and a shortlist of MCP tools. The *Grader and Validator* module in Figure 8 is then used to select the final Strategist supervision. It first removes structurally invalid candidates, such as outputs with missing fields, invalid formats, or unavailable tool names. The remaining candidates are then compared by an LLM-based grader using a rubric prompt that considers attacker-intent correctness, engagement-strategy coherence, and tool relevance. The highest-scored candidate is retained for each stage-level sample, resulting in 300 selected Strategist outputs.

The selected Strategist outputs are combined with the corresponding MCP tool specifications to form Planner-side inputs. The same four-teacher process is applied again to generate 1200 Planner candidates. Each candidate contains a normalized plan and concrete tool arguments. The Planner-side *Grader and Validator* first checks schema validity, including valid tool names, required arguments, argument types, and plan format. Valid candidates are then ranked by an LLM-based grader using a rubric prompt that emphasizes argument completeness, consistency with the stage context, and avoidance of redundant or contradictory tool calls. One Planner output is retained for each sample. The final dataset therefore contains 300 feature-label pairs: each feature consists of the engagement strategy, selected tools, and relevant MCP tool specifications, while each label is the validated Planner output. Table 4 summarizes the sample counts at each stage of the offline supervision pipeline.

Model preparation differs across the two reasoning stages. Strategist-side supervision is used to produce structured high-level outputs and to support Planner data construction, but no

Table 5: Planner supervised fine-tuning configuration

Item	Setting
Base model	Phi-4
Training framework	Unsloth
Fine-tuning method	QLoRA-style supervised fine-tuning
Chat template	phi-4
Max sequence length	16000
Base model loading	4-bit
Trainable parameters	LoRA adapter only
Precision	bfloat16
LoRA rank / alpha / dropout	16 / 32 / 0.05
Training batch size	2
Gradient accumulation steps	4
Learning rate	2.0×10^{-5}
Epochs	2
Optimizer	AdamW 8-bit
Learning-rate scheduler	Cosine
Warmup ratio / weight decay	0.03 / 0.01
Random seed	42
Best-model selection	Lowest validation loss
Dataset split	240 train / 30 validation / 30 test

Strategist SFT is performed in the implemented pipeline. This is because the Strategist mainly performs flexible semantic reasoning over attacker behavior, while the Planner must repeatedly generate schema-compliant tool parameters in a stable format. Therefore, only Planner-side supervision is exported as the SFT target.

The finalized Planner dataset contains 300 samples in total and is split into 240 training samples, 30 validation samples, and 30 test samples. The Planner is fine-tuned using supervised fine-tuning with QLoRA: the base Phi-4 model is loaded in 4-bit precision, while only LoRA adapter parameters are trained and saved. This setting reduces GPU memory usage while adapting the Planner to the bounded MCP action space and structured output format. The main fine-tuning configuration is summarized in Table 5. Although this dataset scale is sufficient for adapting the Planner to the implemented MCP tools, larger-scale supervision may further improve generalization to unseen attack scenarios and environment configurations.

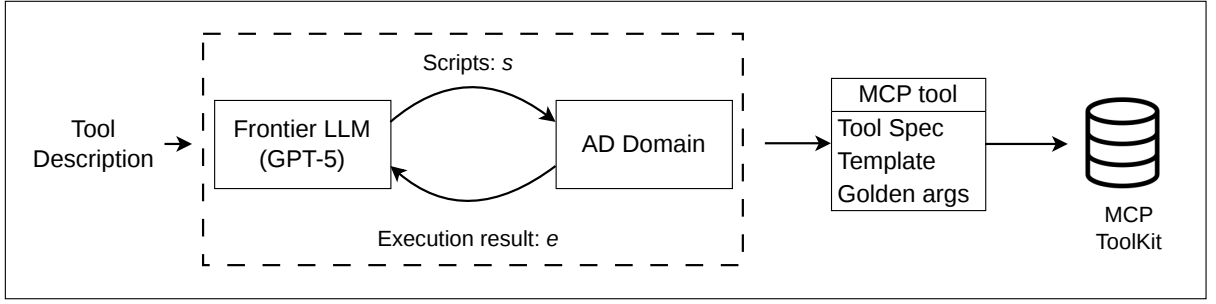


Figure 9: Semi-automated generation and validation process for MCP tools.

5.3 MCP Toolkit Generation and Execution

Figure 9 shows the build-and-validate process used to generate MCP tools for the implemented system. MCP toolkit generation begins from manually designed defender capabilities derived from selected MITRE Engage activities and the modification opportunities available on different host roles in the AD-domain honeynet. For each host type, the defender first identifies environment modifications that are meaningful for attacker engagement. For example, Linux clients and service hosts can expose decoy files, breadcrumbs, and credential-like artifacts; the domain controller can support modifications related to accounts, groups, and privilege relationships; and Windows clients can expose user-context artifacts such as fake recent-file lists and cached credentials. These capabilities support engagement behaviors such as exposing decoy identities, planting artifacts, creating credential surfaces, leaving breadcrumbs, and inducing controlled failure. Each capability is then defined as one MCP tool with a natural-language tool description.

After the tool-level capability is specified, a frontier LLM (GPT-5) is used to generate the corresponding tool specification, Ansible template, and golden arguments. The golden arguments provide a reference input for rendering the template and testing the generated playbook in the real AD environment. If the execution succeeds, the tool is admitted into the toolkit. If execution fails, the execution result is fed back to the LLM for debugging and revision. In this way, MCP toolkit generation is implemented as a semi-automated build-and-validate process that reduces manual engineering effort while keeping the resulting tools executable and extensible.

After validation, each tool is stored in a bounded registry with its name, description, target

host, input schema, template binding, and golden arguments. This registry gives the Planner a stable interface for generating tool calls, while allowing the MCP module to validate arguments and render the corresponding Ansible template at runtime. As a result, every tool exposed to the Planner has a defined interface, an execution target, and a tested deployment path. The finalized toolkit contains 35 MCP tools across 8 functional groups, as summarized in Table ??.

Runtime MCP execution is implemented as an in-process module inside the backend pipeline rather than as a standalone server process. Given a structured plan P , the backend invokes the MCP module with the tool name and parameter values in each plan step. The MCP module validates the arguments against the tool schema, binds them to the corresponding Jinja2-based Ansible template, and renders the final playbook. The playbook is then executed through a subprocess call to the Ansible command-line tool, which connects to the target hosts through SSH or WinRM and applies the requested modifications. This path keeps runtime execution traceable from Planner-generated tool calls to concrete changes in the AD-domain honeynet.

5.4 Attacker Interaction Simulation

Attacker simulation is implemented as an LLM-driven interactive command loop rather than a fixed replay script. Figure 10 illustrates the end-to-end simulation loop. At step i , the attacker model receives the recent session memory, including previous commands and observed responses, together with stage-guided command templates. The Attacker LLM, realized by GPT-5 accessed through the OpenAI API, then generates the next command and structured metadata describing the intended action. The command is executed through a persistent SSH session connected to the defender-side proxy, and the response from the LLM honeynet is written back into memory for the next step. This loop continues until the command budget is reached or the attacker terminates the session.

The use of a persistent SSH session allows the simulated attacker to interact with the honeynet as a continuous shell session. Because multiple commands share the same shell stream, sentinel markers are inserted around each command execution to recover the corresponding output segment and return code. This design preserves command-response boundaries while allowing attacker behavior to evolve from prior observations and react to environment changes

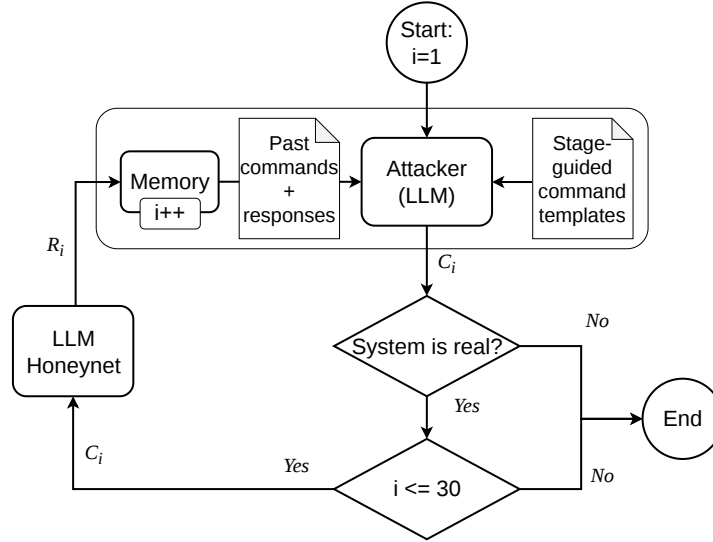


Figure 10: End-to-end attacker interaction simulation loop.

introduced during the session.

The Attacker LLM is guided by a seven-stage post-compromise attack progression: discovery, collection, credential access, privilege escalation, lateral movement, exfiltration, and impact. This progression is a Cyber Kill Chain-style linear abstraction used to support the stage-progression objective $s_i \in \{1, \dots, 7\}$ defined in Chapter 3, rather than a direct reproduction of the full MITRE ATT&CK tactic order. The stages are derived from the ATT&CK tactics most relevant to the implemented AD-domain honeynet: discovery covers host, service, and identity reconnaissance; collection covers data gathering; credential access covers credential discovery and reuse; privilege escalation covers attempts to obtain higher permissions; lateral movement covers cross-host access; exfiltration covers data extraction; and impact covers disruptive or destructive end goals. Tactics that mainly occur before honeynet entry, such as reconnaissance, resource development, and initial access, are omitted because the attacker starts from a pre-established SSH entry session. Defense evasion, persistence, and command-and-control are not modeled as separate stages because the evaluation focuses on interactive post-compromise progression rather than long-term stealth or malware lifecycle management.

At each step, the model selects the next attack direction from the current session history and then generates a command for that direction. To make the simulation resemble a credible attacker, the prompt requires the model to rely on evidence discovered in the environment, recognize repeated unrealistic responses as possible honeypot signals, pursue lateral movement

when useful credentials or host information are found, and treat exfiltration or impact as late-stage objectives. For stages that are difficult to execute reliably through free-form generation, such as privilege escalation, lateral movement, exfiltration, and impact, the prompt provides reference command templates. These templates guide command construction without fixing a predetermined attack path, preserving open-ended behavior while reducing unnecessary trial and error.

In summary, the implemented Engage-Agent system integrates a Paramiko-based MITM SSH frontend, a fine-tuned Phi-4 Planner with QLoRA adapters, an Ansible-based MCP execution module covering 35 tools, and an LLM-driven attacker simulator. The next chapter evaluates this implementation under the experimental setup described in Chapter 3.



Chapter 6. Experimental Evaluation

6.1 Experimental Setup

This chapter evaluates the proposed Engage-Agent system, denoted as **Ours (Two-layer LLM Honeynet(FT))** in this chapter. The evaluation uses three evidence sources. The main evidence comes from controlled attacker experiments, which compare end-to-end attacker engagement across different systems. A fixed-scenario planner benchmark is then used to analyze the internal quality of generated plans. Finally, online deployment observations provide supplementary context about the SSH entry point in a real exposed setting. These three sources answer different questions and are interpreted separately.

Table 6 summarizes the systems compared in the controlled attacker experiments. Ours is the full proposed system with Strategist LLM, fine-tuned Planner LLM, MCP execution, and AD-domain environment modification. Two-layer LLM Honeynet and One-layer LLM Honeynet are ablations of the proposed architecture. LLM-based Honeypot represents reply-level LLM deception, while QRASSH represents the traditional SSH honeypot baseline. The baselines are implemented to reproduce the interaction mechanisms relevant to this study, but may not replicate every engineering detail of the original systems.

The controlled attacker experiment uses the LLM-driven attacker simulator described in Chapter 5. Each system is evaluated over 30 sessions, and each session has a maximum budget of 30 attacker commands. The attacker uses GPT-5-mini with temperature 1.0, generates one command at each step, and conditions its next action on up to 30 recent interaction steps. Commands are generated online from session history rather than replayed from stored scripts, so this experiment measures end-to-end engagement under free-form simulated attacker interaction.

The planner benchmark is separate from the end-to-end attacker experiment. It uses five fixed scenario categories shared with the SFT pipeline, but uses separately generated command

Table 6: Compared systems in the controlled attacker experiments

System	Environment	Main mechanism
Ours (Two-layer LLM Honeynet(FT))	AD-domain honeynet	Strategist LLM, fine-tuned Planner LLM, and MCP-based environment modification.
Two-layer LLM Honeynet	AD-domain honeynet	Strategist LLM with non-fine-tuned Planner LLM.
One-layer LLM Honeynet	AD-domain honeynet	Single LLM performs both engagement decision and planning.
LLM-based Honeypot	Single-host reply setting	LLM directly generates attacker-visible shell replies without backend environment modification.
QRASSH	SSH honeypot	Traditional SSH honeypot baseline.

traces rather than SFT training samples or labels. Each scenario contains 24 commands and is divided into three 8-command stages. The benchmark compares only planner configurations: Ours (Two-layer LLM Honeynet(FT)), Two-layer LLM Honeynet, and One-layer LLM Honeynet. Online deployment observations are collected from SSH proxy logs over a 35-day deployment period and are used only as supplementary operational context for the exposed SSH entry point.

6.2 Attack Progression and Behavioral Diversity

This section evaluates whether each system can guide the simulated attacker toward deeper and more diverse behavior. Attack depth is measured by the maximum attack stage reached in each episode. For each generated command, the attacker LLM also outputs a structured attack-stage label. The label must belong to one of the seven predefined stages: discovery, collection, credential access, privilege escalation, lateral movement, exfiltration, and impact. The episode depth is then computed as the highest stage weight reached in that session. Behavioral diversity is measured by unique techniques. For this metric, all attacker commands and command outputs in one session are given to a GPT-based evaluator, which maps each step to a technique from a predefined technique pool. The metric is the number of techniques with non-zero counts in that episode. Failed commands are included because they still reflect attacker behavior.

Figure 11 compares the maximum attack stage depth across the controlled attacker sessions. The single-host baselines remain much shallower: QRASSH reaches an average depth

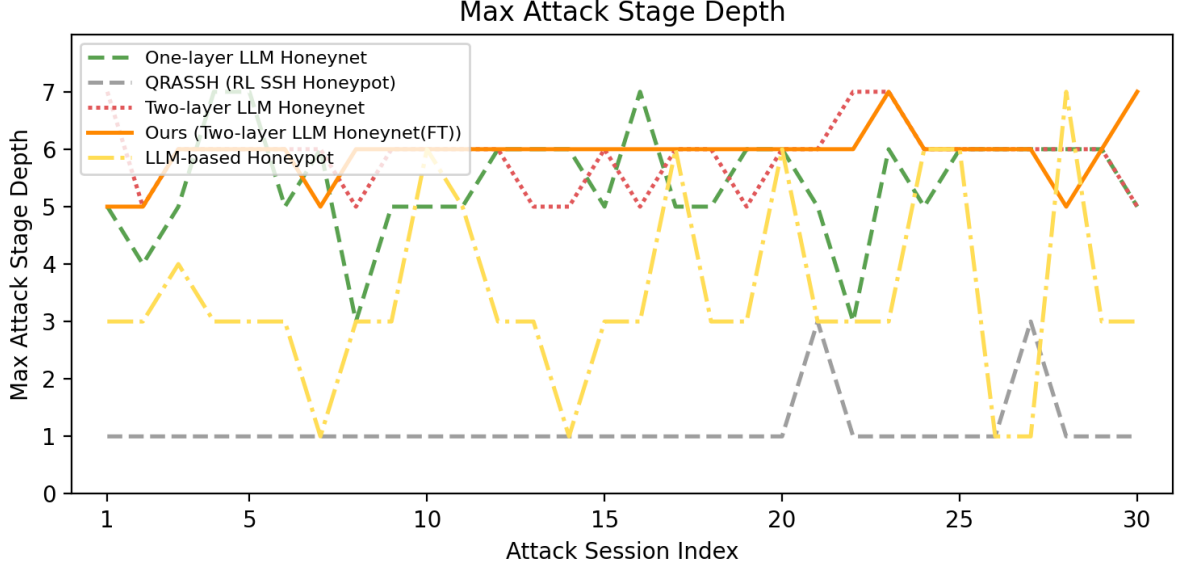


Figure 11: Maximum attack stage depth across 30 controlled attacker sessions. Higher values indicate deeper simulated attack progression.

of 1.13, and the LLM-based Honeypot reaches 3.47. In contrast, the honeynet systems reach substantially deeper stages. The One-layer LLM Honeynet reaches an average depth of 5.43, the Two-layer LLM Honeynet reaches 5.87, and Ours (Two-layer LLM Honeynet(FT)) reaches the highest average depth of 5.93. These results show that the AD-domain honeynet provides attacker-visible paths toward credential access, privilege escalation, lateral movement, and later-stage objectives that are difficult to support in single-host baselines.

The gap between honeynet systems and single-host baselines is larger than the gap among the honeynet variants. This suggests that the multi-host AD environment is a major factor behind deeper progression. Ours improves the average depth over the One-layer LLM Honeynet by approximately 9%, but this improvement is smaller than the baseline-to-honeynet gap. These results suggest that the large improvement in attack depth mainly comes from the multi-host honeynet environment. The two-layer and fine-tuned design further improves the average depth, but the difference among honeynet variants is smaller than the gap between honeynet systems and single-host baselines. Therefore, we treat the architectural effect as supporting evidence rather than as the primary explanation for deeper progression.

Figure 12 compares behavioral diversity. QRASSH produces an average of 2.43 unique techniques per episode, and the LLM-based Honeypot produces 4.43. The honeynet systems produce much broader behavior: One-layer LLM Honeynet reaches 11.07, Two-layer LLM

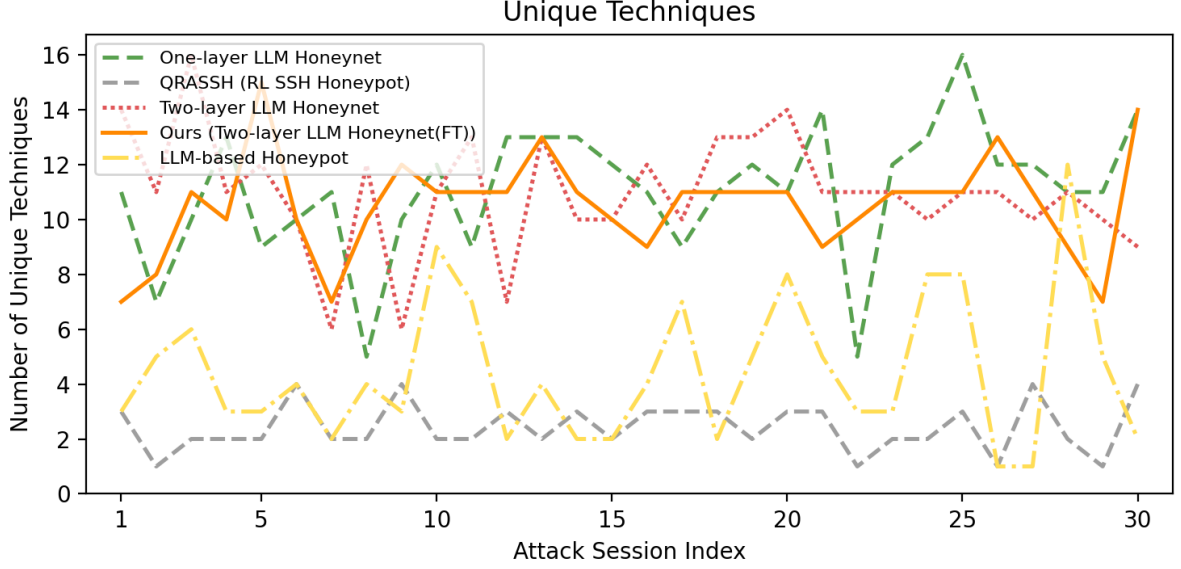


Figure 12: Number of unique techniques observed across 30 controlled attacker sessions. Higher values indicate broader behavioral coverage.

Honeynet reaches 10.96, and Ours reaches 10.53. Thus, the honeynet variants produce more than four times the diversity of QRASSH and more than twice the diversity of the LLM-based Honeypot.

These results indicate that real multi-host state creates more opportunities for varied attacker behavior than single-host interaction or reply-level generation. The three honeynet variants remain in the same high-diversity range, suggesting that behavioral diversity is mainly driven by the shared environment. At the same time, Ours achieves the highest attack depth but slightly lower diversity. Under the fixed 30-command budget, this is expected: when the attacker follows one effective path more deeply, fewer commands remain for exploring unrelated techniques. Thus, the diversity result shows that all honeynet variants expose broad attack opportunities, while the depth result shows that Ours better supports deeper progression along a coherent attack path.

6.3 Interaction Realism and Cross-Host Behavior

This section evaluates whether the systems provide believable interaction and whether they support cross-host attacker behavior. Perceived realism is computed from individual command responses. Each evaluated item consists of an attacker command, stdout, stderr, and exit code. A

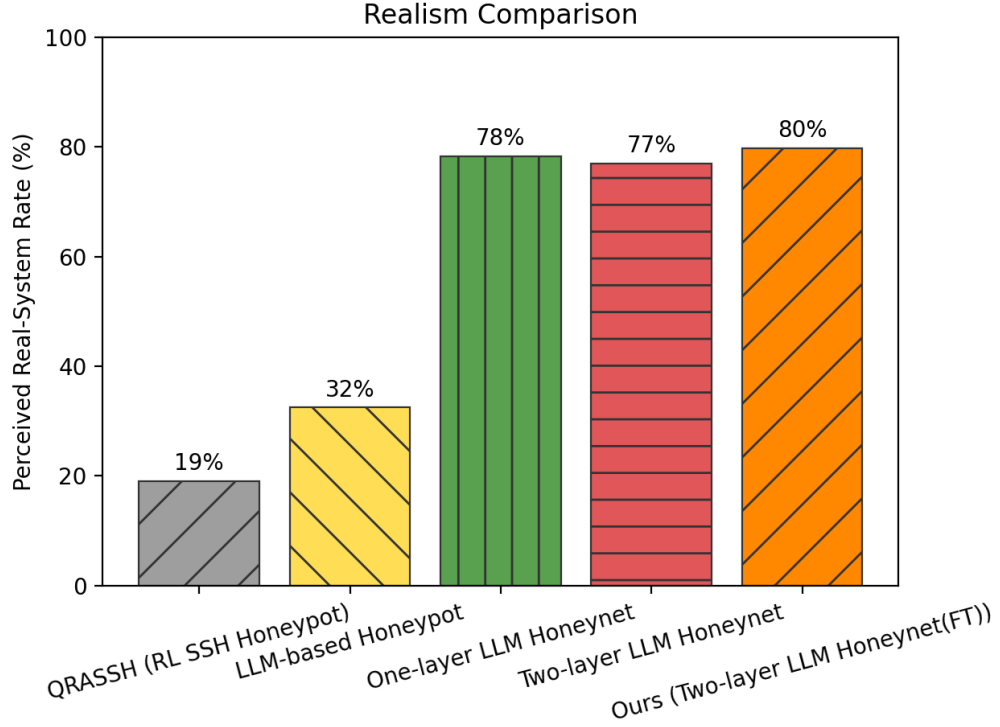


Figure 13: Perceived Realism Rate in the controlled attacker experiments. Higher values indicate that command responses are judged as more plausible real-system interaction.

GPT-based evaluator labels the response as *realistic*, *synthetic*, or *inconclusive* with an evidence strength. These judgments are mapped to realism scores and averaged across all command responses for each system to obtain the Perceived Real-System Rate. Therefore, the metric reflects the realism of observed system responses, not a binary session-level success label. Cross-host behavior is measured by lateral movement attempts and success rate. Attempts are identified by rule-based matching of remote-access tools and protocols such as SSH, SCP, SFTP, WinRM, RDP, SMB, WMI, PsExec, Telnet, and Impacket-based commands. A successful attempt is counted when the command exits successfully and its output does not contain failure patterns such as authentication failure, connection refusal, timeout, or permission denial.

Figure 13 compares the Perceived Realism Rate across systems. QRASSH reaches 19%, and the LLM-based Honeypot reaches 32%. In contrast, the honeynet systems achieve much higher realism: One-layer LLM Honeynet reaches 78%, Two-layer LLM Honeynet reaches 77%, and Ours reaches 80%. Ours is therefore more than four times higher than QRASSH and about 2.5 times higher than the LLM-based Honeypot.

The three honeynet systems are all close to 80%, suggesting that the main realism gain

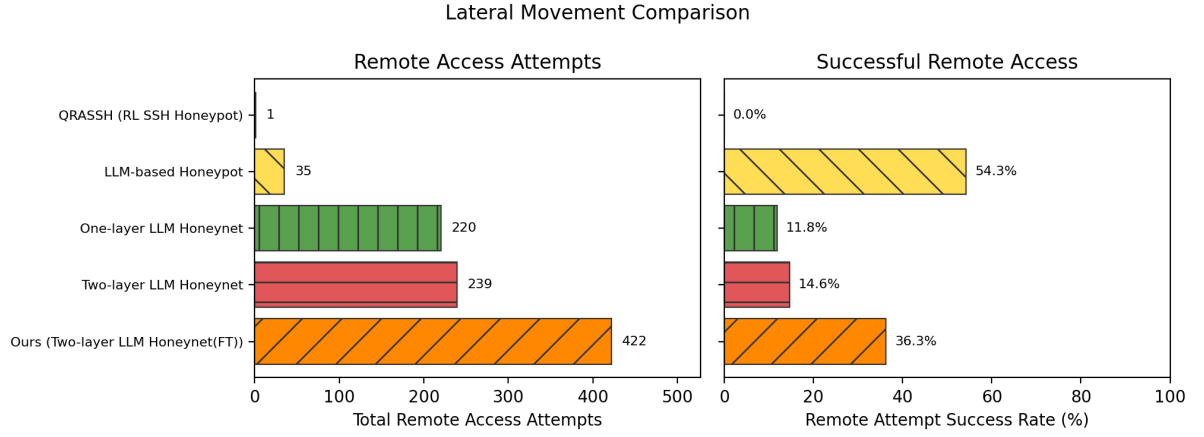


Figure 14: Lateral movement attempts and success rates in the controlled attacker experiments. Attempt volume and success rate should be interpreted jointly.

comes from environment-level interaction and multi-host consistency rather than from the LLM architecture alone. The LLM-based Honeypot can generate plausible replies, but those replies are not grounded in persistent files, accounts, services, or host relationships. In contrast, the honeynet systems expose a real multi-host environment, so later attacker observations are produced by actual system state.

Figure 14 compares lateral movement behavior. Ours produces 422 remote access attempts with a 36.3% success rate, which is the strongest combination of attempt volume and success among the evaluated systems. The One-layer and Two-layer LLM Honeynet variants also produce many more attempts than the single-host baselines, with 220 attempts at 11.8% success and 239 attempts at 14.6% success, respectively. QRASSH produces only one attempt with no success.

The LLM-based Honeypot shows a 54.3% success rate, but this is based on only 35 attempts and must be interpreted carefully. The same rule-based success criterion is applied to all systems; however, for a reply-generation baseline, a successful-looking output can be generated rather than produced by real cross-host execution. Ours combines a much larger number of attempts with a substantial success rate grounded in the AD-domain honeynet. This indicates stronger practical support for realistic lateral movement under the controlled attacker protocol.

6.4 Planner Output Quality Benchmark

The previous sections show that the multi-host honeynet is the dominant factor behind the large gap between honeynet systems and single-host baselines. However, the end-to-end results alone are not sufficient to isolate the effect of the LLM-agent architecture, because the honeynet variants share the same AD-domain environment and therefore achieve similar attack depth and behavioral diversity. The role of the LLM agent is to decide which environment modifications should be applied, instantiate the required tool arguments, and produce a coherent engagement plan. Therefore, this section evaluates the generated plans directly under fixed scenario-stage inputs. This benchmark is a component-level evaluation of Planner output quality rather than a measurement of end-to-end attacker engagement.

The benchmark compares One-layer LLM Honeynet, Two-layer LLM Honeynet, and Ours (Two-layer LLM Honeynet(FT)). It uses the same five high-level scenario categories as the SFT pipeline, but uses separately generated fixed traces. Therefore, the benchmark should be interpreted as an in-domain component benchmark rather than an out-of-distribution generalization test. Each stage has human-authored expected tools. Success Rate measures whether the Planner produces parseable, valid steps using known tools. Expected Tool F1 measures whether the Planner selects the expected defender tools for a given stage; higher values indicate better tool selection. Critical Argument Score measures whether the generated tool arguments contain the important values needed for deployment, such as target hosts, accounts, credentials, file paths, or service-specific parameters. Consistency Score measures whether the arguments across multiple tool calls form a coherent engagement plan and remain consistent with the scenario context. Both scores are evaluated by an LLM judge and adjusted with deterministic penalties for weak alignment, copying, or duplicate behavior; higher values indicate better plan quality. Duplicate Tool Rate is the fraction of stages where the selected tool list contains repeated tools, so lower values are better. Composite Score aggregates these metrics with larger weights on argument quality and consistency:

$$S_{\text{comp}} = 0.10S_{\text{succ}} + 0.10S_{\text{tool}} + 0.35S_{\text{arg}} + 0.35S_{\text{cons}} + 0.10(1 - R_{\text{dup}}).$$

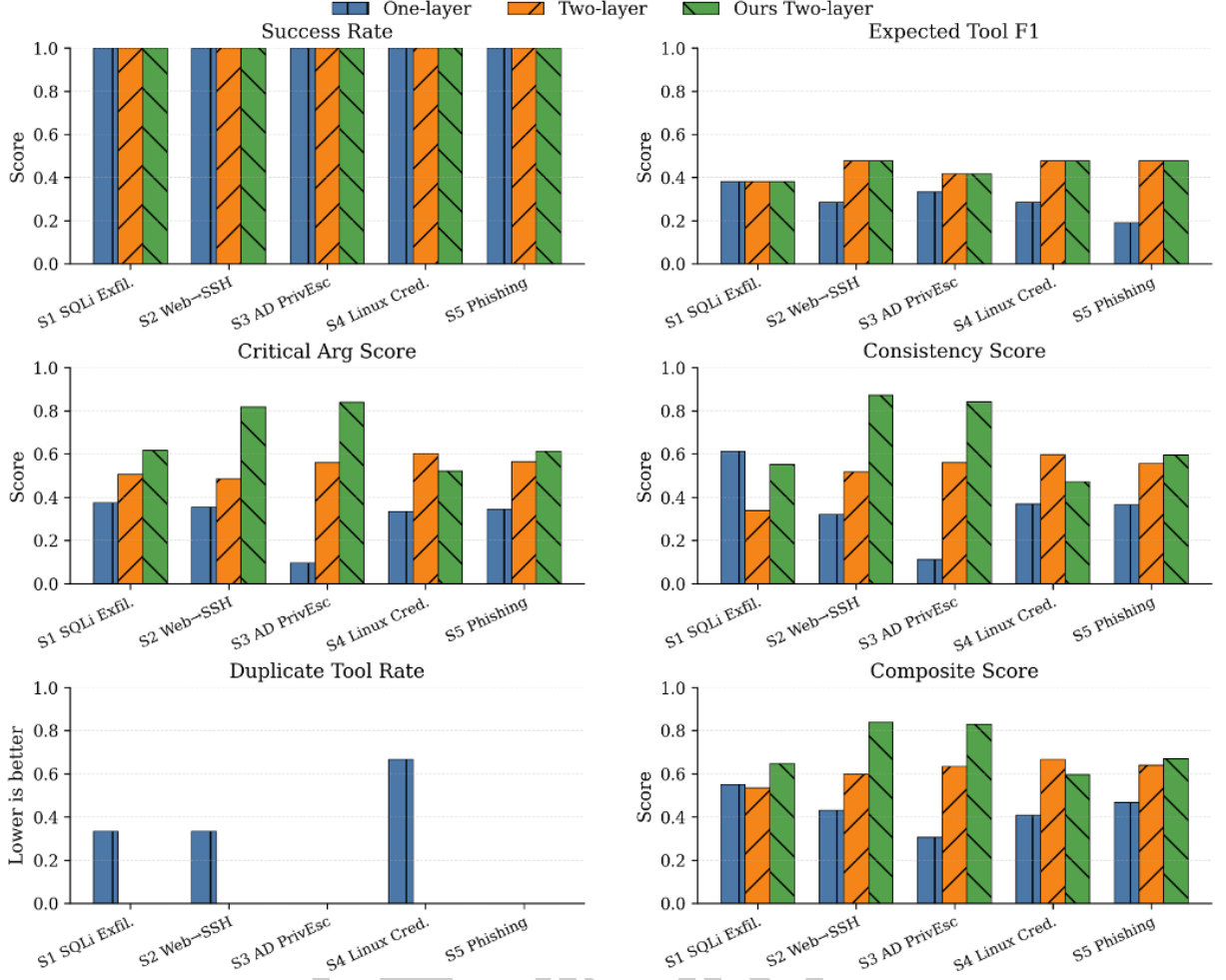


Figure 15: Planner output quality comparison under fixed scenario-stage inputs. The benchmark evaluates internal plan quality rather than end-to-end attacker engagement.

where S_{succ} , S_{tool} , S_{arg} , and S_{cons} denote success, expected-tool matching, critical argument quality, and consistency scores, respectively, and R_{dup} denotes the duplicate-tool rate.

Figure 15 shows the planner benchmark results. All three configurations achieve a success rate of 1.0, meaning that they can produce parseable and executable plans in this benchmark. However, success alone does not distinguish plan quality, because an executable plan can still contain weak tool choices, missing arguments, inconsistent parameters, or redundant tool calls. The more informative metrics are therefore tool matching, argument quality, consistency, duplicate-tool rate, and composite score.

The composite score improves from 0.43 for the One-layer LLM HoneyNet to 0.61 for the Two-layer LLM HoneyNet. Ours further improves the score to 0.72, which is about 67% higher than the one-layer design and about 18% higher than the non-fine-tuned two-layer variant. This

shows that separating strategy reasoning from tool-argument planning improves plan quality, and that Planner fine-tuning further improves the generated tool parameters.

The detailed metrics explain the source of this improvement. The one-layer design shows weaker argument quality and consistency, and it also produces repeated tool usage in several scenarios. This suggests that asking one model to perform attacker interpretation, tool selection, and concrete argument generation in a single step makes the output less stable. The two-layer configurations largely remove duplicate-tool behavior, and Ours further improves argument quality and cross-tool consistency. Thus, the planner benchmark complements the end-to-end experiments by showing that the FT two-layer design improves the internal quality of the environment-modification plans.

6.5 Online Deployment Observations

This section provides supplementary observations from deploying the SSH proxy as an exposed entry point. These observations are not controlled experiments and are not used to compare the full adaptive honeynet against baselines. They only provide operational context for the type and volume of traffic collected by the exposed SSH service.

Figure 16 shows the daily SSH audit activity during the deployment period. Daily activity is computed from proxy audit events, including login attempts, execution requests, interactive sessions, and binary-upload sessions. The proxy received continuous login attempts after deployment, with several days reaching hundreds of attempts. This shows that the exposed entry point was visible enough to attract non-trivial real SSH traffic. Among post-authentication events, execution requests dominate, while interactive shell sessions appear much less frequently. This pattern suggests that most observed post-authentication activity came from automated SSH clients or scripted command execution rather than long manual sessions.

Figure 17 summarizes the observed command behavior. Commands are grouped into rule-based categories such as host reconnaissance, malware staging or execution, writable-directory probing, network or lateral probing, shell enablement, session termination, and binary payload upload. Host reconnaissance is the most frequent category, indicating that observed commands commonly inspect the host, environment, and available execution context. Malware staging

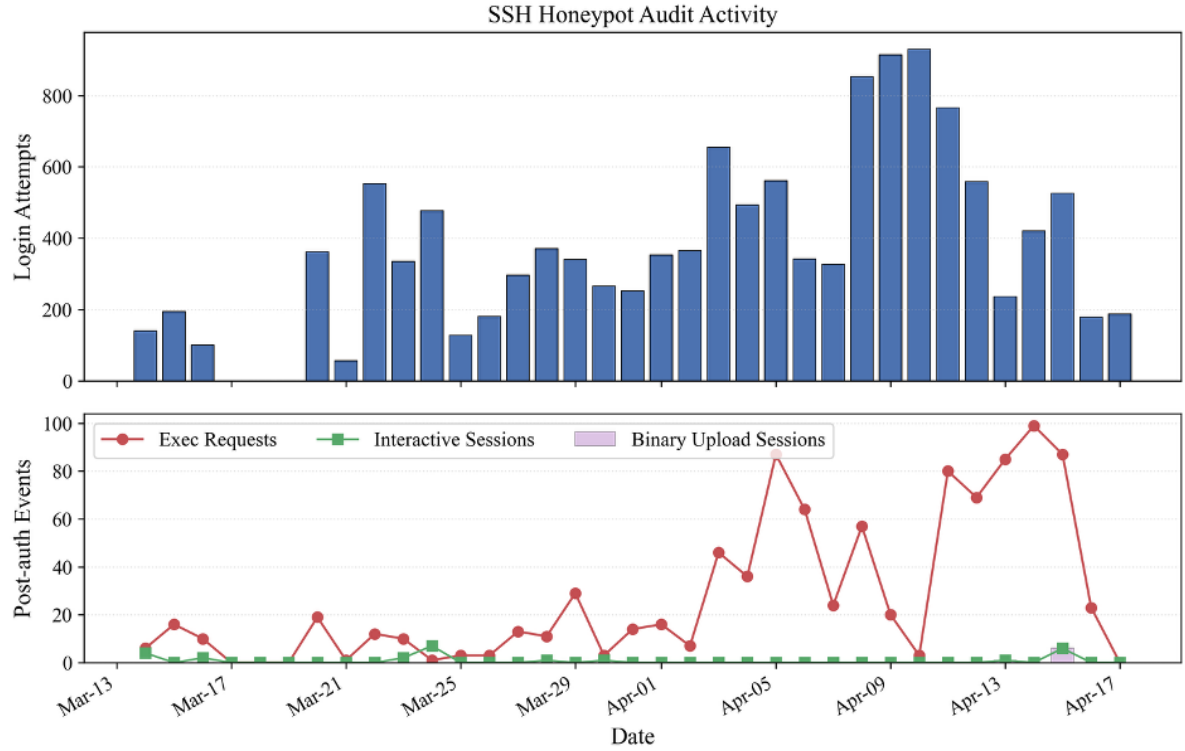


Figure 16: Online SSH audit activity observed at the deployed proxy entry point.

and execution is the second major behavior, showing that many sessions quickly move from probing to preparing or running payloads. Together with the dominance of exec requests, this behavior distribution is consistent with automated SSH attack traffic that performs short reconnaissance and payload-staging commands after authentication. Binary payload upload occurs less frequently, but it contributes a much larger logged volume when it appears.

These observations support the operational relevance of collecting command-level SSH activity, especially reconnaissance, payload preparation, and post-compromise execution attempts. However, because the data is collected at the proxy layer and appears to be dominated by automated or scripted behavior, it should be interpreted only as supplementary deployment context rather than as evidence of full adaptive honeynet engagement with sophisticated human attackers.

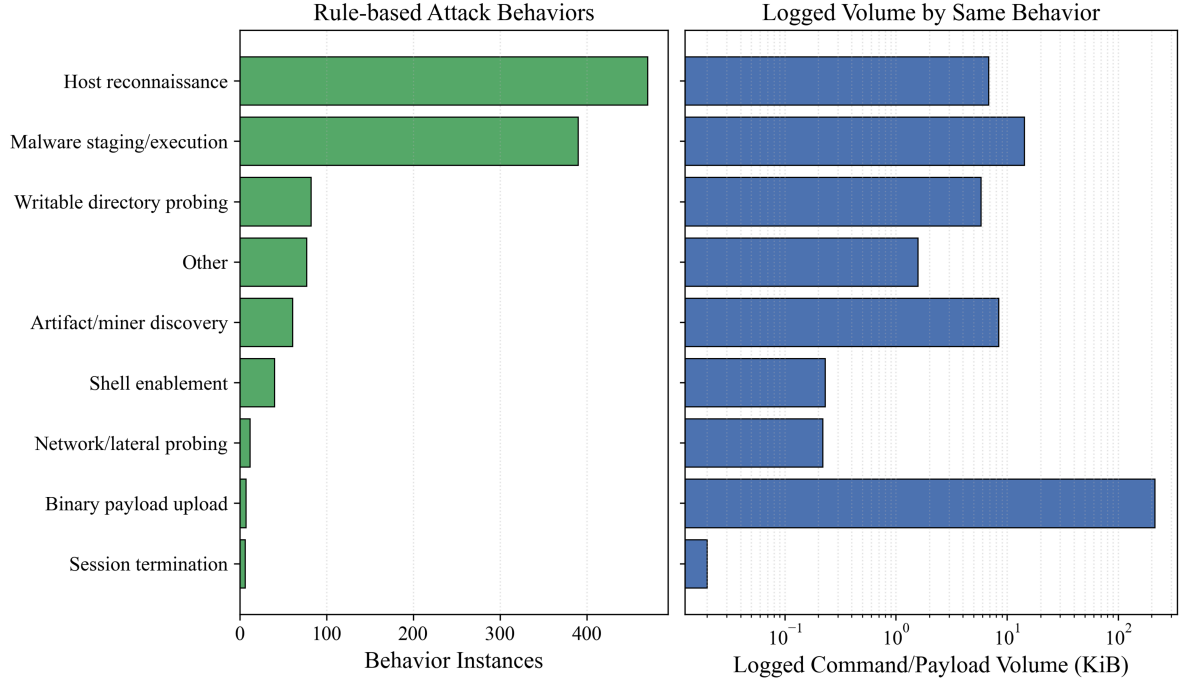


Figure 17: Observed command categories and logged volume from online SSH proxy traffic.

6.6 Discussion and Limitations

The results suggest that Engage-Agent benefits from two complementary factors: the multi-host AD honeynet and the LLM agent that adapts it. The shared honeynet environment provides realistic attack surfaces, persistent host state, and cross-host relationships, which explains why all honeynet-based systems substantially outperform the single-host baselines in attack depth, behavioral diversity, and perceived realism. The LLM agent contributes at a different layer: it decides which environment modifications to apply in response to attacker behavior and how to instantiate those modifications through concrete tool arguments. This role is most visible in the lateral-movement result and the planner benchmark. Ours (Two-layer LLM Honeynet(FT)) produces the strongest cross-host behavior in the controlled attacker experiment, while the benchmark shows that the two-layer and fine-tuned design reduces redundant tool use and improves argument quality and plan consistency. Overall, the results support the thesis that environment-driven deception can create the conditions for deeper simulated attacker engagement, while the LLM agent improves the reliability and adaptiveness of the actions used to shape that environment.

This evaluation has several limitations. First, the attacker in the controlled experiments is an implemented LLM-based simulator rather than a human attacker, so the measured attack depth and behavioral diversity depend on the simulator’s prompt design, model capability, and session logic. Second, environment modification is an indirect engagement mechanism: the system can plant files, accounts, credentials, services, or cross-host clues, but the attacker must still discover and use them before they affect behavior. Third, because the implemented attacker may not fully exploit subtle differences among honeynet variants, the end-to-end results of One-layer, Two-layer, and Ours can appear close even when their generated plans differ in quality. Finally, the results are specific to the implemented AD-domain honeynet, selected tool set, session budget, and evaluation protocol, and should therefore be interpreted within this experimental scope rather than as a universal claim for all enterprise environments or attacker types.



Chapter 7. Conclusions and Future Work

This thesis addressed the problem of adaptive attacker engagement in honeynet environments. Modern enterprise attacks often involve multi-stage behaviors such as credential reuse, privilege escalation, lateral movement, and Active Directory compromise. Traditional single-host honeypots and reply-generation-based deception systems are limited in their ability to support such cross-host attack progression. To address this limitation, this thesis proposed Engage-Agent, an adaptive honeynet engagement framework that combines a real AD-domain honeynet with an LLM-based decision agent. Instead of directly fabricating attacker-visible replies, the system modifies the underlying environment so that attacker observations emerge from real system state.

The proposed system integrates an SSH interception frontend, a two-layer LLM agent, an MCP-based execution toolkit, and a multi-host AD-domain honeynet. The Strategist LLM analyzes recent attacker commands, infers attacker intent, and selects appropriate engagement tools. The Planner LLM then converts the selected strategy and tools into structured tool arguments, which are rendered and executed through MCP and Ansible-based environment modification. Through this design, the system separates high-level attacker understanding from execution-oriented planning, while keeping all environment changes bounded by predefined and validated tool interfaces.

The experimental results show that environment-grounded deception improves simulated attacker engagement. Compared with single-host baselines, the honeynet-based systems achieved deeper attack progression, broader behavioral diversity, and higher perceived realism. The results also show that Ours (Two-layer LLM Honeynet(FT)) produced the strongest lateral-movement behavior among the evaluated systems. While the shared multi-host AD environment explains much of the improvement over single-host baselines, the planner benchmark shows that the two-layer and fine-tuned design improves internal plan quality, argument correctness, consistency, and duplicate-tool avoidance. These findings support the central claim of

this thesis: realistic multi-host state provides the foundation for deeper engagement, while the LLM agent improves how the defender adapts that environment during interaction.

This thesis makes three main contributions. First, it presents an adaptive honeynet framework that bridges environment realism and adaptive attacker interaction. Second, it introduces an environment-driven deception approach in which attacker-visible behavior is produced through real system modification rather than direct reply generation. Third, it develops a bounded LLM-to-tool execution pipeline, including the Strategist–Planner architecture, MCP tool registry, and Planner fine-tuning process, for translating high-level engagement decisions into executable multi-host environment changes. Together, these contributions provide a practical foundation for building adaptive, coordinated, and realistic honeynet engagement systems.

Several limitations remain. The controlled evaluation relies on an LLM-driven attacker simulator rather than human attackers, so the observed attack depth and behavioral diversity depend on the simulator’s prompt design, model capability, and session logic. In addition, environment modification is an indirect engagement mechanism: the system can plant files, credentials, accounts, services, or cross-host clues, but the attacker must still discover and use them before they affect behavior. The evaluation is also conducted in one AD-domain honeynet with a specific tool set and scenario design, and the planner benchmark should be interpreted as an in-domain component evaluation rather than a broad generalization test.

Future work can extend Engage-Agent in several directions. First, the evaluation should include stronger attacker models and human red-team studies to better understand how adaptive environment modification affects real adversaries. Second, the defender can be extended with feedback-aware adaptation, allowing the system to reason about whether an attacker noticed or used a deployed modification before selecting the next action. Third, the MCP toolkit can be expanded to cover more enterprise services, cloud resources, and deception techniques beyond the current AD-domain environment. Finally, larger-scale deployment, stronger statistical analysis, and human assessment of realism would provide a more comprehensive evaluation of adaptive honeynet engagement in practical settings.

References

- [1] T. Yang, C. Zhang, and Y. Xu, “Honeyfactory: A honeynet generation architecture for capturing multi-stage network attacks,” *Electronics*, vol. 13, no. 2, p. 361, 2024.
- [2] L. J. Möller, “An adaptive multi-layered honeynet architecture for threat behavior analysis via deep learning,” arXiv preprint arXiv:2512.07827, 2025.
- [3] A. Pauna, A.-C. Iacob, and I. Bica, “Qrassh: A self-adaptive ssh honeypot driven by q-learning,” in *Proc. IEEE International Conference on Communications (ICC)*, 2018, pp. 1–6.
- [4] Z. Wang, J. You, H. Wang, T. Yuan, S. Lv, Y. Wang, and L. Sun, “Honeygpt: Breaking the trilemma in honeypots with large language models,” arXiv preprint arXiv:2406.01882, 2025.
- [5] MITRE Corporation, “Mitre att&ck: Get started,” <https://attack.mitre.org/resources/>, 2024, accessed: 2026-04-16.
- [6] —, “Mitre engage,” <https://engage.mitre.org/>, 2026, accessed: 2026-04-16.
- [7] C.-Y. Hsieh, “Active directory deception: A reinforcement learning honeypot aligned with mitre engage,” Master’s thesis, National Yang Ming Chiao Tung University, 2025.
- [8] F. Lucchese, F. Zago, M. Polino, and S. Zanero, “Honeyics: A high-interaction honeynet for industrial control systems,” in *Proc. International Conference on Availability, Reliability and Security (ARES)*, 2023.
- [9] M. e. a. Kouremetis, “Mirage: Cyber deception against autonomous cyber attacks in emulation and simulation,” *Annals of Telecommunications*, vol. 79, pp. 803–817, 2024.

- [10] R.-H. Hwang, J.-C. Huang, Y.-C. Lai, and Y.-D. Lin, “Leveraging reinforcement learning for llm-based generic honeypots with mitre engage framework,” Master’s thesis, National Yang Ming Chiao Tung University, 2025.
- [11] S. Liu, J. Sun, and K. Sun, “Catch me if you can: Effective honeypot placement in dynamic ad attack graphs,” George Mason University, Fairfax, VA, USA, Tech. Rep., 2024.
- [12] D. Blazakis, J. S. Lee, and J. A. Harder, “Honeywin: High-interaction windows honeypot in enterprise environment,” in *Proc. IEEE European Symposium on Security and Privacy Workshops (EuroS&PW)*, 2020, pp. 236–243.
- [13] J. Sun, S. Liu, and K. Sun, “A scalable high fidelity decoy framework against sophisticated cyber attacks,” in *Proc. ACM Workshop on Moving Target Defense (MTD)*, 2019, pp. 37–46.
- [14] Y. Aoike, M. Kamizono, M. Eto, N. Matsumoto, and N. Yoshida, “Decoy-file-based deception without usability degradation,” in *Proc. IEEE Asia-Pacific Conference on Computer Science and Data Engineering (CSDE)*, 2021, pp. 971–978.
- [15] V. S. N. Tinnaluri and N. Shaik, “A comprehensive approach: Developing a honeypot system to thwart cyber attackers,” *Educational Administration: Theory and Practice*, vol. 30, no. 5, pp. 9093–9099, 2024.
- [16] Y.-D. Lin, Y.-H. Lu, R.-H. Hwang, Y.-C. Lai, D. Sudyana, and W.-B. Lee, “Evolving ml-based intrusion detection: Cyber threat intelligence for dynamic model updates,” *IEEE Transactions on Machine Learning in Communications and Networking*, vol. 3, pp. 605–612, 2025.
- [17] O. Rehnäck, “A case study of unauthorized login attempts against honeypots via remote desktop,” Master’s thesis, Luleå University of Technology, 2023.
- [18] S. Suratkar, K. Shah, A. Sood, A. Loya, D. Bisure, U. Patil, and F. Kazi, “An adaptive honeypot using q-learning with severity analyzer,” *Journal of Ambient Intelligence and Humanized Computing*, vol. 13, pp. 4865–4876, 2022.

- [19] N. Lee, T. Wattanawong, S. Kim, K. Mangalam, S. Shen *et al.*, “Llm2llm: Boosting llms with novel iterative data enhancement,” arXiv preprint arXiv:2403.15042, 2024.
- [20] J. Chen, X. Han, Y. Ma, X. Zhou, and L. Xiang, “Unlock the correlation between supervised fine-tuning and reinforcement learning in training code large language models,” arXiv preprint arXiv:2406.10305, 2024.
- [21] A. Pareja, N. S. Nayak, H. Wang *et al.*, “Unveiling the secret recipe: A guide for supervised fine-tuning small llms,” arXiv preprint arXiv:2412.13337, 2024.
- [22] M. Nădaș, L. Dioșan, and A. Tomescu, “Synthetic data generation using large language models: Advances in text and code,” arXiv preprint arXiv:2503.14023, 2025.
- [23] S. Pujar, L. Buratti, X. Guo *et al.*, “Automated code generation for information technology tasks in yaml through large language models,” arXiv preprint arXiv:2305.02783, 2023.
- [24] L. Fan, Z. Liu, H. Wang, L. Bao, X. Xia, and S. Li, “Fgit: Fault-guided fine-tuning for code generation,” arXiv preprint arXiv:2503.16913, 2025.
- [25] H. Li, W. Feng, X. Zhou, and Z. Shen, “Gift: Gibbs fine-tuning for code generation,” in *Proc. Annual Meeting of the Association for Computational Linguistics (ACL)*, 2025, pp. 12 271–12 285.
- [26] X. Jiang, Y. Dong, Z. Jin, and G. Li, “Seed: Customize large language models with sample-efficient adaptation for code generation,” arXiv preprint arXiv:2403.00046, 2024.